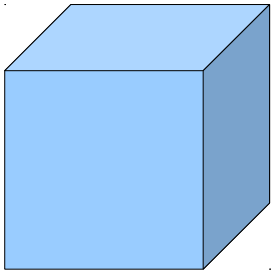
UNIVERSITÄT LEIPZIG
Institut für Wirtschaftsinformatik

# Entwicklung verteilter Anwendungen

## Nebenläufigkeit

**Pascal Kovacs**

16. April 2020



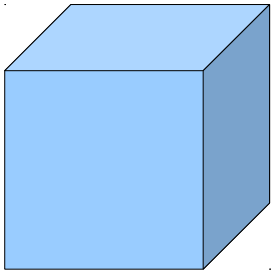
Java

# Ziele Java Grundlage

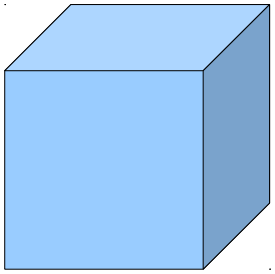
- Java als Sprache
- Anweisungen, Operatoren
- Objektorientierung, Vererbung und Interfaces
- Exceptionhandling, Collections, Wrapperklassen

# Kontrollfragen

- Was sind die Designziele der Sprache Java?
- Was ist der Unterschied zwischen primitiven und Referenz-Datentypen?
- Wozu dient der instanceof Operator?
- Wie funktioniert Vererbung in Java?
- Was sind Java Interfaces, welche Bestandteile können sie enthalten?
- Was ist die Klasse Object?
- Wie sind Strings in Java umgesetzt und wie wird mit ihnen gearbeitet?
- Welche zwei Arten von Exceptions gibt es in Java?
- Welche zwei Möglichkeiten der Behandlung von Exceptions gibt es in Java?
- Was ist der Unterschied zwischen Collections und Arrays?



Java



Nebenläufigkeit

# Ziele Nebenläufigkeit

- Unterschied serielle/parallele Verarbeitung verstehen
- Motivation & Herausforderungen nebenläufiger Verarbeitung kennen
- Begriffe & Techniken nebenläufiger Systeme kennen
- Probleme nebenläufiger Verarbeitung verstehen

# Kontrollfragen

- Was sind die Herausforderungen und Ziele paralleler Verarbeitung?
- Was ist die Aufgabe des Schedulers im Kontext von Multitasking?
- Was ist der Unterschied zwischen Prozessen und Threads
- Was ist eine Race Condition und wie kann sie verhindert werden?
- Was sind die Probleme nebenläufiger Systeme und welche Lösungen gibt es?
- Was sind die Probleme, die wiederum durch diese Lösungen entstehen?

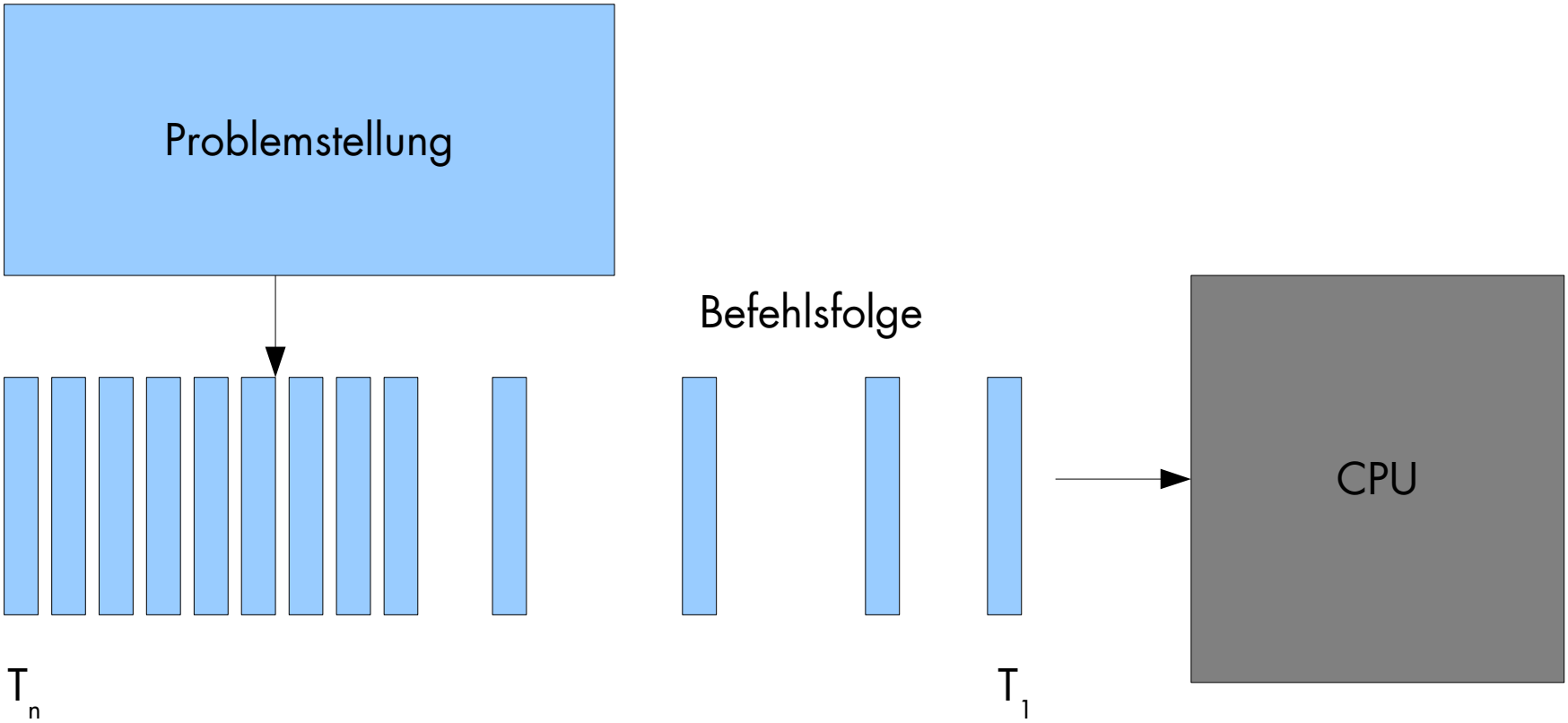
# Übersicht

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

# Serielle Verarbeitung

- Im *Normalfall* wird ein Programm seriell abgearbeitet
- Problemstellung wird in diskrete Abfolge von Instruktionen zerteilt
- Abarbeitung der Befehle erfolgt nacheinander
- Ausführung erfolgt auf einer einzelnen Recheneinheit
- Je Zeiteinheit wird lediglich ein Befehl ausgeführt/abgearbeitet

# Serielle Verarbeitung




T - Ausführungszeitpunkt

# Beispiel Berechnung Primzahlen

- Es sollen alle Primzahlen kleiner 1.000.000 ermittelt werden
- Test auf Primzahl erfolgt durch Suche möglicher Teiler pro Zahl
- Ergibt die Teilung den Rest 0 handelt es sich um keine Primzahl
- Weiter mit der nächsten Zahl
- Sind alle möglichen Teiler untersucht worden ist es eine Primzahl

- Sequentielle Abarbeitung erfolgt durch nacheinander ausgeführte Untersuchung aller ungerader Zahlen, beginnend mit 3

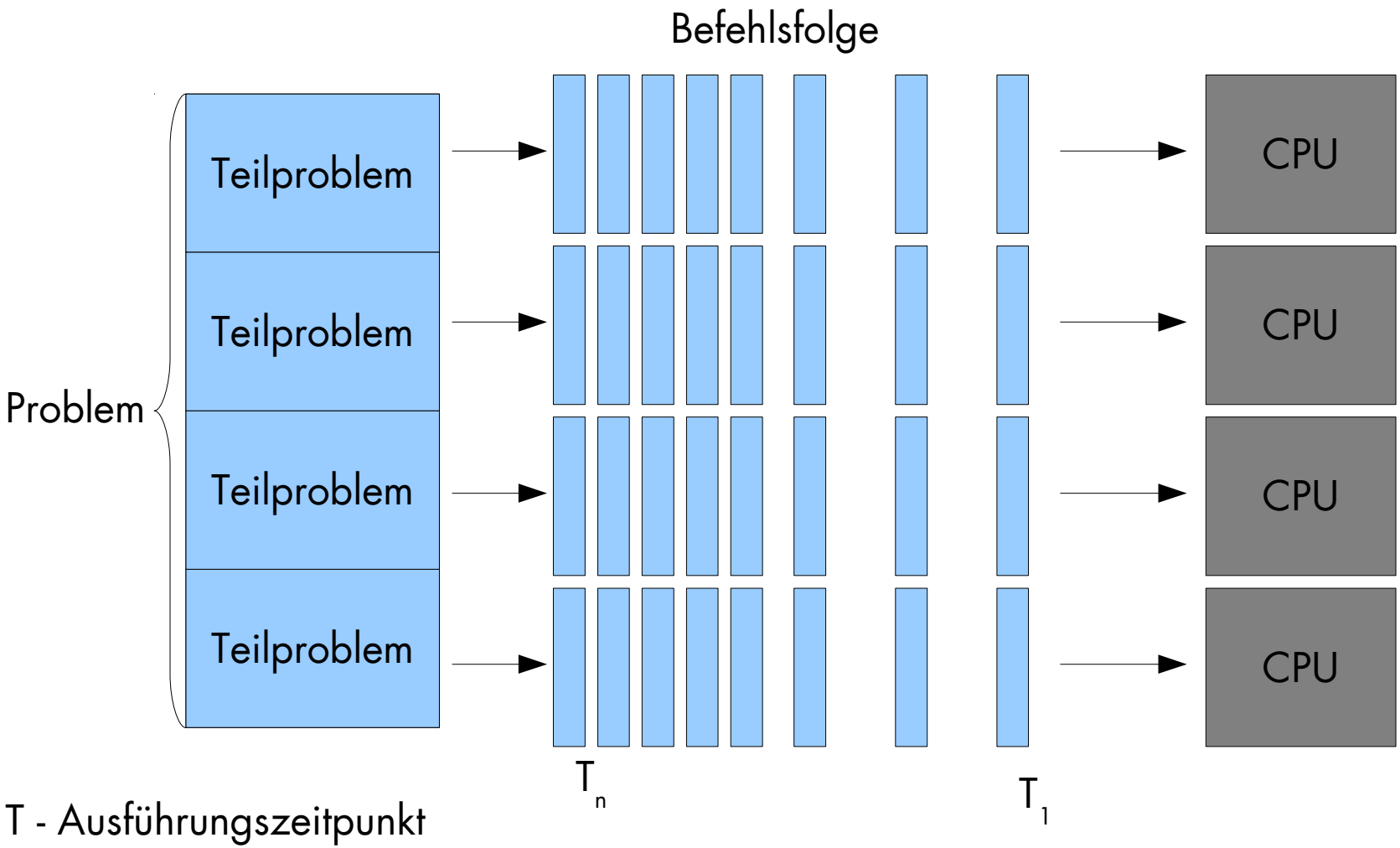
# PseudoCode

```
zahl = 3
while ( zahl < 1000000) {
   teiler = 3
   while ( teiler < zahl ) {
      if ( zahl mod teiler == 0 ) {
         break
      } else {
         teiler = teiler + 2
      }
   }
   if ( teiler >= zahl ){
      primzahlen[] = zahl
   }
   zahl = zahl + 2
}
```

# Parallele Verarbeitung

- ...ist mehrere Befehle zur gleichen Zeit ausführen
- Erfolgt durch gleichzeitige Nutzung mehrerer Prozessoren/Rechner
- Problem wird in diskrete Teilprobleme zerlegt
- Herunterbrechen der Teilprobleme auf Serie von Instruktionen
- Ausführung auf mehreren Recheneinheiten(Prozessoren/Rechnern)
- Teilprobleme werden somit parallel gelöst
- Das Problem wird parallel gelöst

# Parallel Verarbeitung



# Beispiel parallele Berechnung von Primzahlen

- Mögliche Problemzerlegung zur Parallelisierung:
- Gleicher Untersuchungsablauf
  - Zahlenbereich wird in 4 gleiche Teile a 250.000 Zahlen zerlegt
  - Es werden parallel 4 Zahlen verarbeitet
- Anderer Untersuchungsablauf
  - Aufteilung des Zahlenbereichs der Teiler
  - Prüfung einer Primzahl wird auf 4 parallele Verarbeitungen verteilt
  - Hypothese, dass Nicht-Primzahlen so schneller gefunden werden
  - Greift vielleicht aber erst bei sehr großen Zahlen

# PseudoCode

```
Thread1.check_prim(3, 250000, primzahlen)
Thread2.check_prim(250000, 500000, primzahlen)
Thread3.check_prim(500000, 750000, primzahlen)
Thread4.check_prim(750000, 1000000, primzahlen)
Wait until all threads finished
```

```
Function check_prim(min, max, primzahlen[]):
  while ( min < max) {
    teiler = 3
    while ( teiler < min ) {
      if ( min  mod teiler == 0 ) {
        break
      } else {
        teiler = teiler + 2
      }
    }
    if ( teiler >= min ){
      primzahlen[] = min
    }
    min = min + 2
  }
```

# Unterscheidung paralleler Verarbeitung

- Gleicher Algorithmus mehrfach parallel mit verschiedenen Daten
  - Funktioniert so lange sich Eingangs- und Ausgangsdaten nicht bedingen
  - Primzahlvariante 1 Aufteilung Zahlenintervall funktioniert
  - Primzahlvariante 2 Aufteilung Teilerintervall funktioniert nicht!
- Identifizieren einzelne Teilaufgaben zur parallelen Verarbeitung
  - Identifikation der einzelnen Teilaufgaben eventuell nicht trivial
  - Koordination der Abhängigkeiten zwischen Aufgaben aufwändig
  - Beispiel eines parallel arbeitenden Systems mit Aufgabentrennung
    - 1) Einlesen und Konvertieren der Daten von einer Datenquelle
    - 2) Verarbeiten der eingelesenen Daten
    - 3) Anzeige der verarbeiteten Daten
    - 4) Export/Logging der verarbeiteten Daten

# Motivation

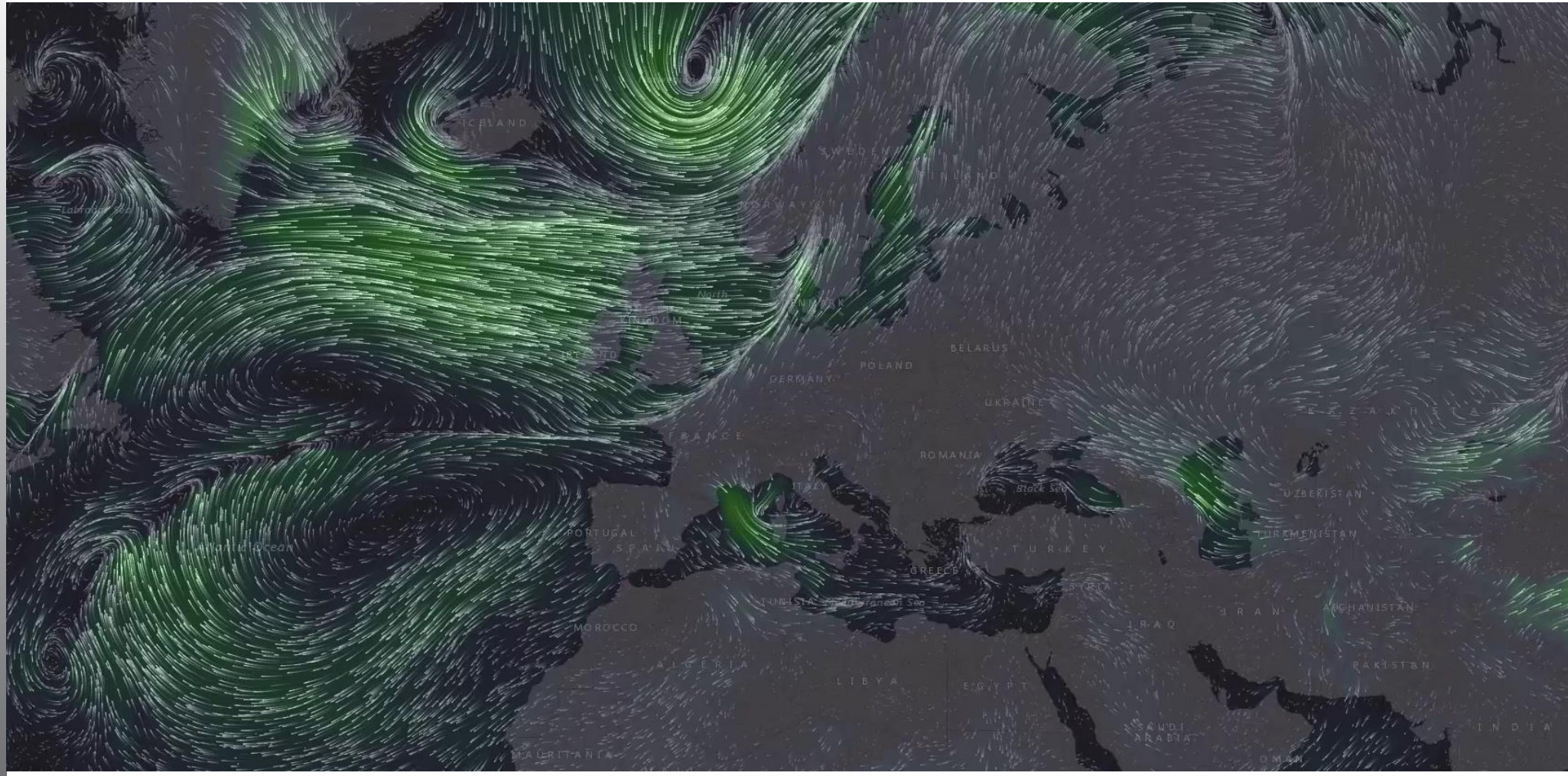
- Serielle Verarbeitung…
- Parallele Verarbeitung…
- Aber wozu?

# Herausforderungen der realen Welt an die IT sind riesig!

# Komplexe Produktionsabläufe

http://www.enisco.de/images/emesgraphic.jpg

# Physikalische Prozesse verstehen/vorhersagen




http://www.freegliders.com/wp-content/uploads/2014/12/Snapshot.jpg

# Analyse gigantischer Datenmengen

http://d152j5tfobgaot.cloudfront.net/wp-content/uploads/2015/02/yourstory_BigData.jpg

# Komplexe Märkte abbilden

https://cleverexpress.files.wordpress.com/2014/07/energie-sparen1.jpg

# Anwendungsfälle für parallele Verarbeitung

- Umfangreiche Datengrundlage
  - Eine Recheneinheit irgendwann immer überfordert (Physikalische Grenze)
- Verarbeitungsgeschwindigkeit nicht akzeptabel
  - Bearbeitung muss in einer bestimmten Zeit erfolgen
- Verfügbarkeit
  - Ein System muss gegen Ausfall geschützt sein
- Usability
  - Arbeiten mit mehreren Aufgaben gleichzeitig notwendig
- Komplexe Problemstellungen
  - Reale Vorgänge sind oft ein Komplex gleichzeitiger Aktivitäten

# Ziele paralleler Verarbeitung

- Verarbeitung in endlicher Zeit erreichen
- Beschleunigung einer Verarbeitung
- Höhere Verfügbarkeit des Systems
- Arbeiten an mehreren Aufgaben ermöglichen
- Komplexe Problemstellung handhaben

# Begriff der Nebenläufigkeit

- Eigenschaft von Systemen, welche mehrere Verarbeitungen parallel ausführen
- Die verschiedenen Verarbeitungen interagieren oder beeinflussen sich dabei gegebenenfalls gegenseitig
- System-theoretisches Modell der parallelen Verarbeitung eines Problems ohne detaillierte Betrachtung der Ausführungsumgebung

# Kehrseite

- Interaktion paralleler Verarbeitung erhöht Anzahl der Ausführungspfade eines Systems → Das Verständnis leidet
- Gemeinsame Nutzung von Ressourcen kann zu Indeterminismus und spontan fehlerhaftem Verhalten von Programmen führen
- Notwendigkeit von koordinierenden Techniken für Ausführung und Ressourcenzuweisung erhöht Realisierungs- und Wartungsaufwand
- Aufgabenzerteilung und -aufteilung in Teilprobleme ist nicht trivial
- Nebenläufige Programmierung ist deutlich komplexer und schwerer zu realisieren (Programmierkonstrukte, Datenabhängigkeiten)

# Negative Folgen

- Mehraufwand für Koordination der Nebenläufigkeit
- Fehler sind schwer zu lokalisieren da oft nicht deterministisch
- System ist insgesamt schwerer verständlich

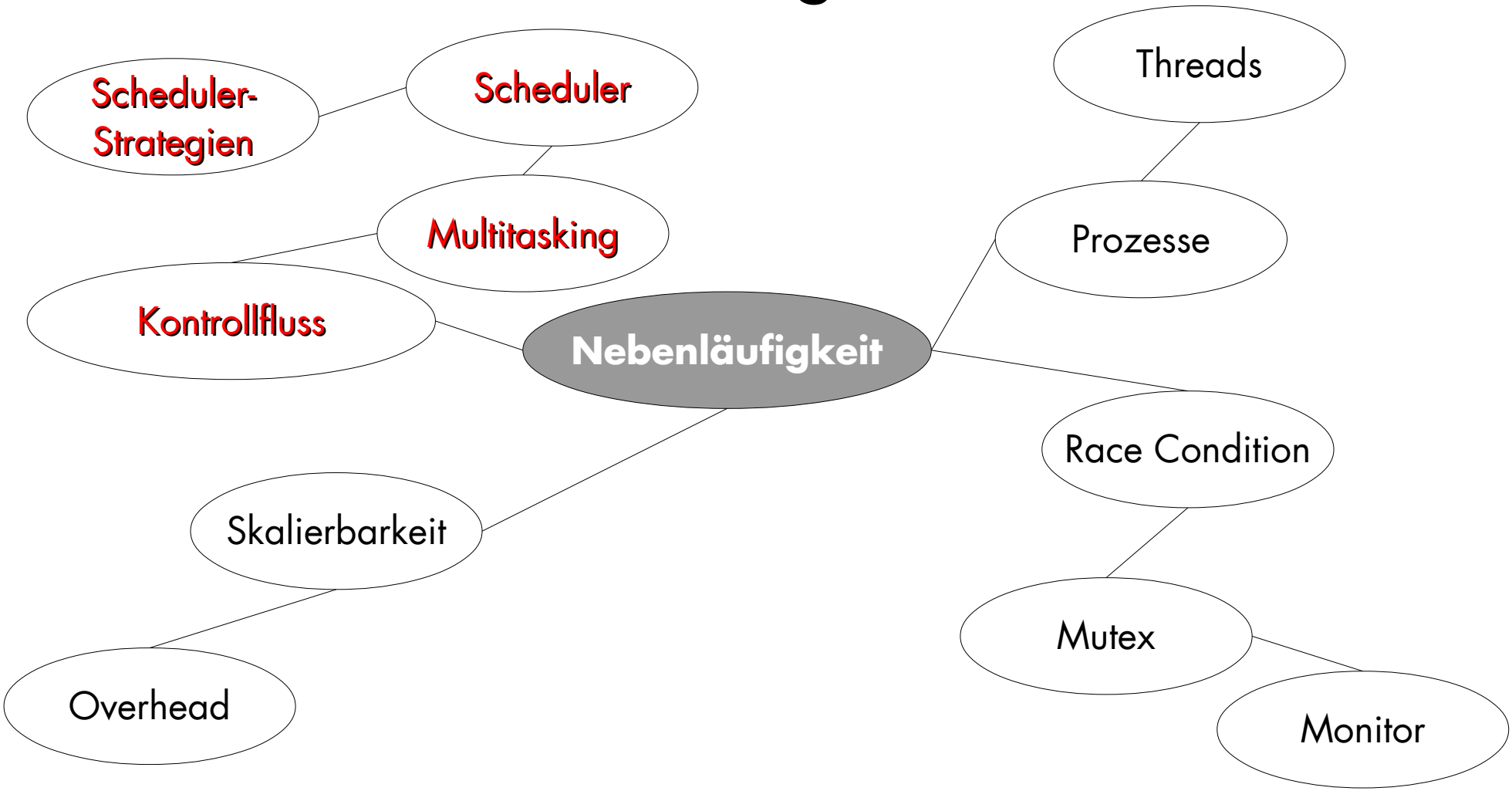
- Höherer Aufwand für programmiertechnische Umsetzung
- Höherer Aufwand für Fehleridentifikation und -Behebung
- Höherer Aufwand bei Service und Wartung

Bei gleichem Umfang und erhöhtem Leistungsdurchsatz (durch Nebenläufigkeit) steigen die Aufwände für die Entwicklung und Qualitätssicherung deutlich an!

# Übersicht

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

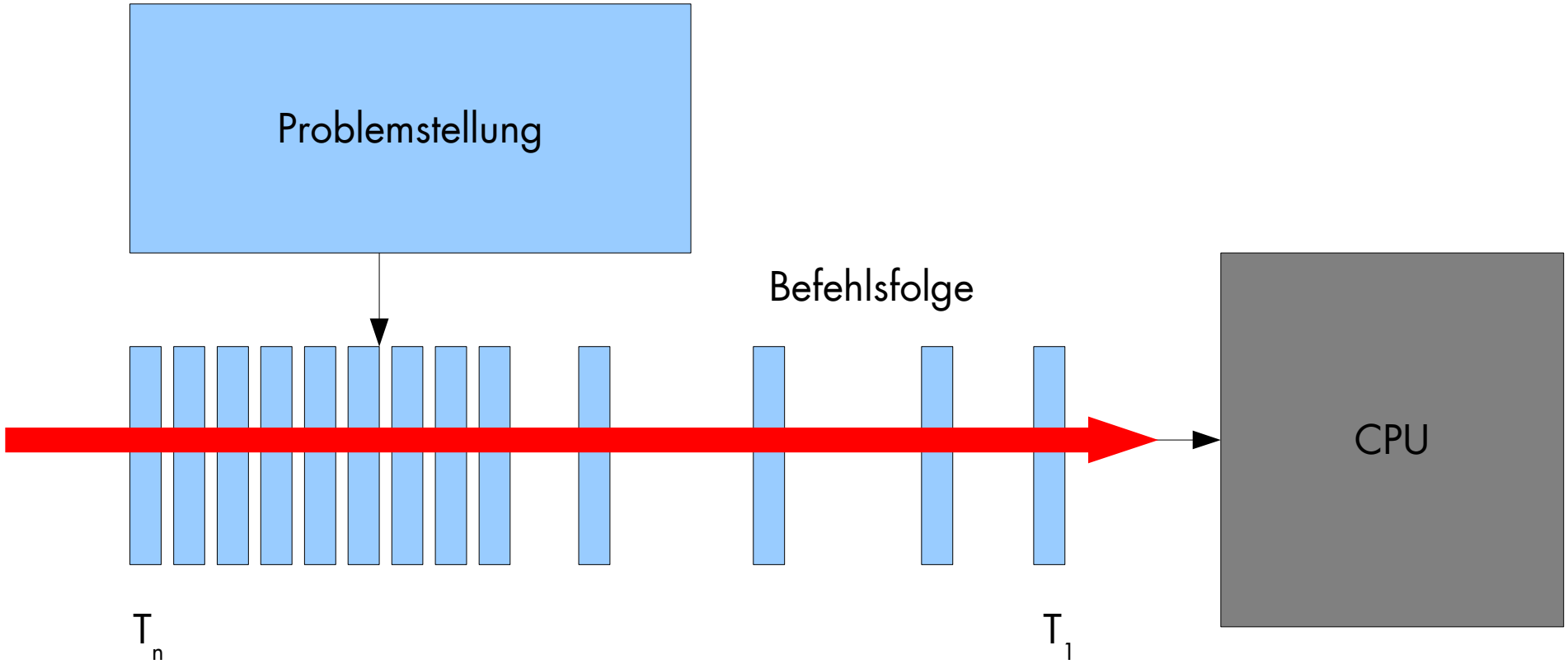
# Kontext der Nebenläufigkeit



# Kontrollfluss

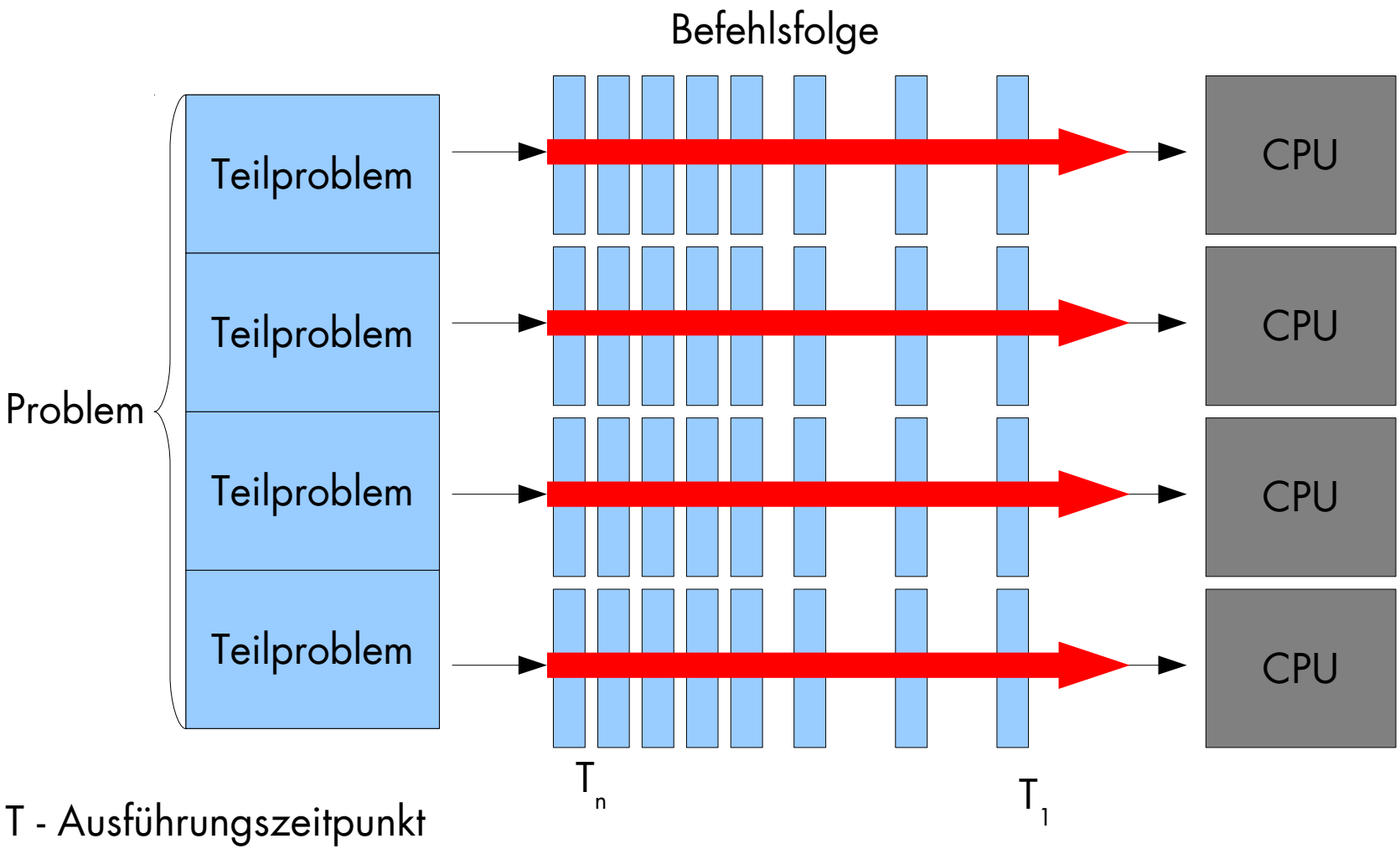
- Einzelner Kontrollfluss
  - Zeitliche Abfolge der einzelnen Befehle eines Computerprogramms
  - In der Regel durch Befehlsreihenfolge vorgegeben
  - Kontrollstruktur erlaubt Abweichen von sequenzieller Abarbeitung
- Mehrere Kontrollflüsse bei paralleler Verarbeitung
  - Erzeugen Abhängigkeiten untereinander
  - Beeinflussen zeitliche Abarbeitung in einzelnen Kontrollflüssen
  - Beeinflussen Reihenfolge der Abarbeitung ähnlich Kontrollstruktur

# Einzelner Kontrollfluss




T - Ausführungszeitpunkt

# Mehrere Kontrollflüsse



# Multitasking

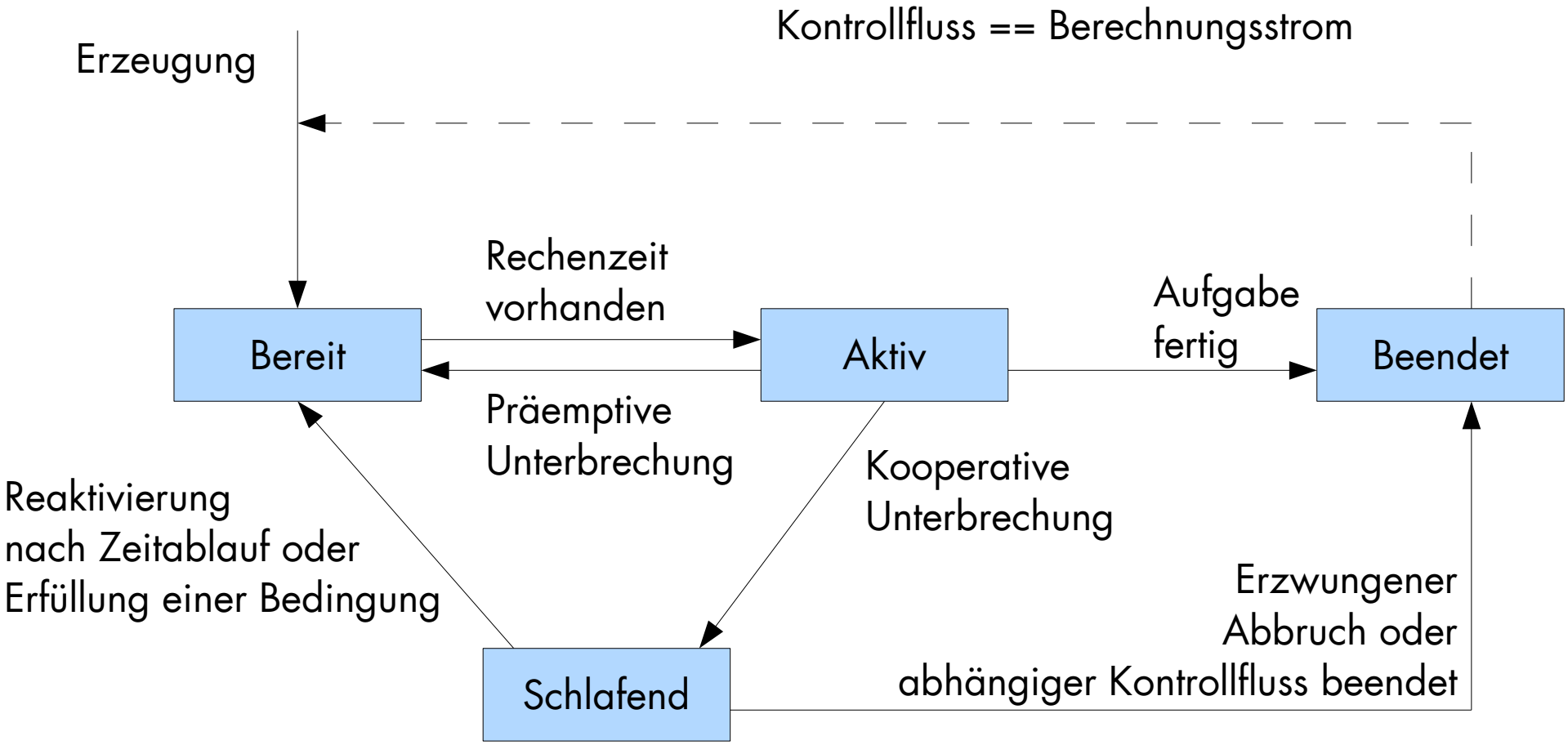
- Pro Recheneinheit nur ein Kontrollfluss gleichzeitig ausführbar
- Häufig jedoch deutlich mehr Kontrollflüsse als Recheneinheit
- Andere Kontrollflüsse müssen warten bis der aktive beendet ist
- Schlecht für Bedienung mehrerer Anwendungen
- Beispiel: Warten beim Kopieren großer Dateien

- Mehr-Prozessor-Systeme gibt es „erst" seit ca. 20? Jahren
- Auch davor schon mehrere Anwendungen gleichzeitig möglich
- Lösung heißt Multitasking bzw. Multithreading

# Multitasking

- Virtueller Rechner für jede Anwendung
- Betriebssystem verteilt und koordiniert Rechenzeit pro Kontrollfluss
- Wechsel zwischen Kontrollflüssen erfolgt in kleinen Zeitabständen
- Dadurch wird ein **scheinbar** paralleler Ablauf erreicht
- Selbst bei Einkernprozessoren möglich
- Ein Kontrollflusslebenszyklus ermöglicht Multitasking

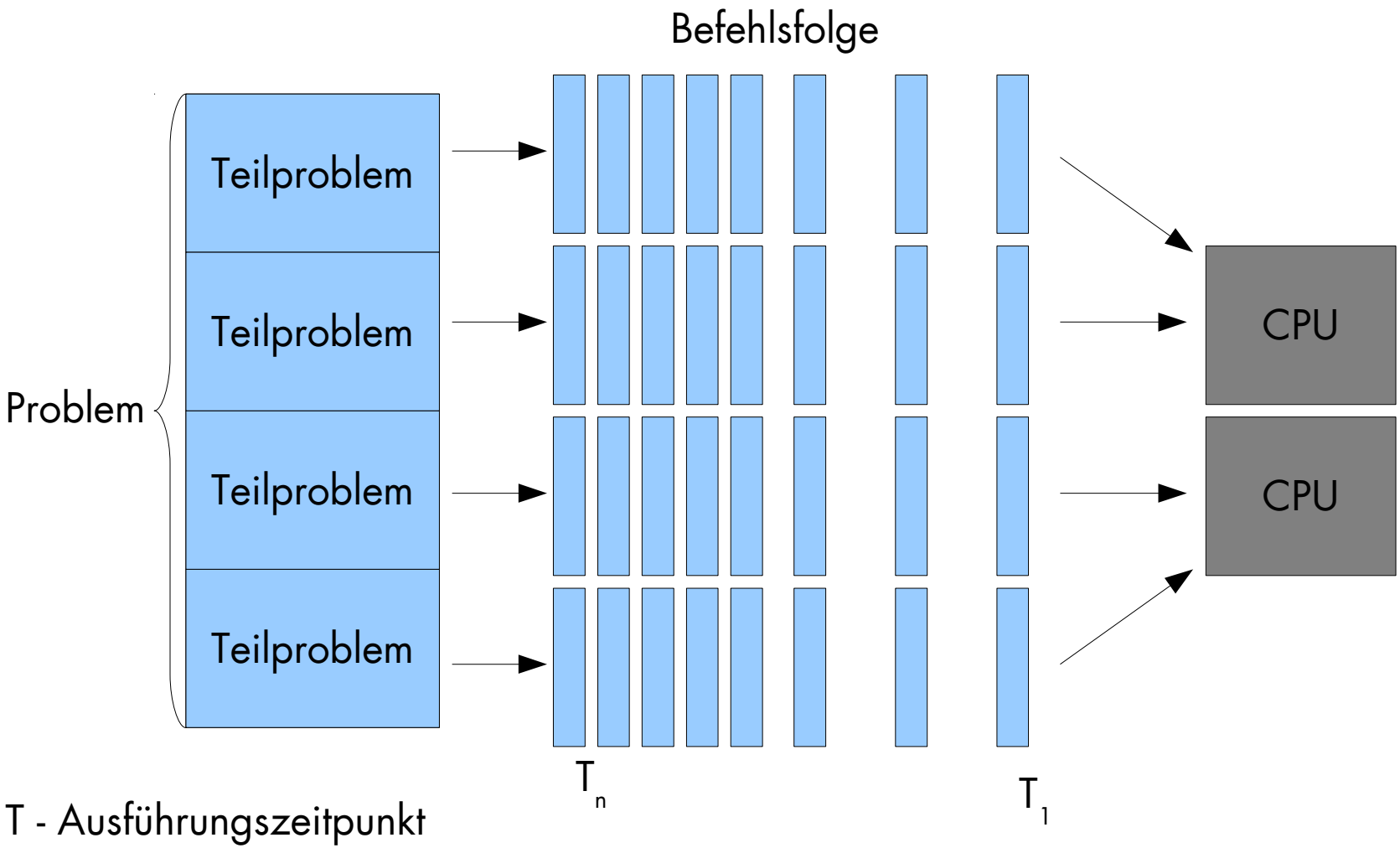
# Kontrollflusslebenszyklus



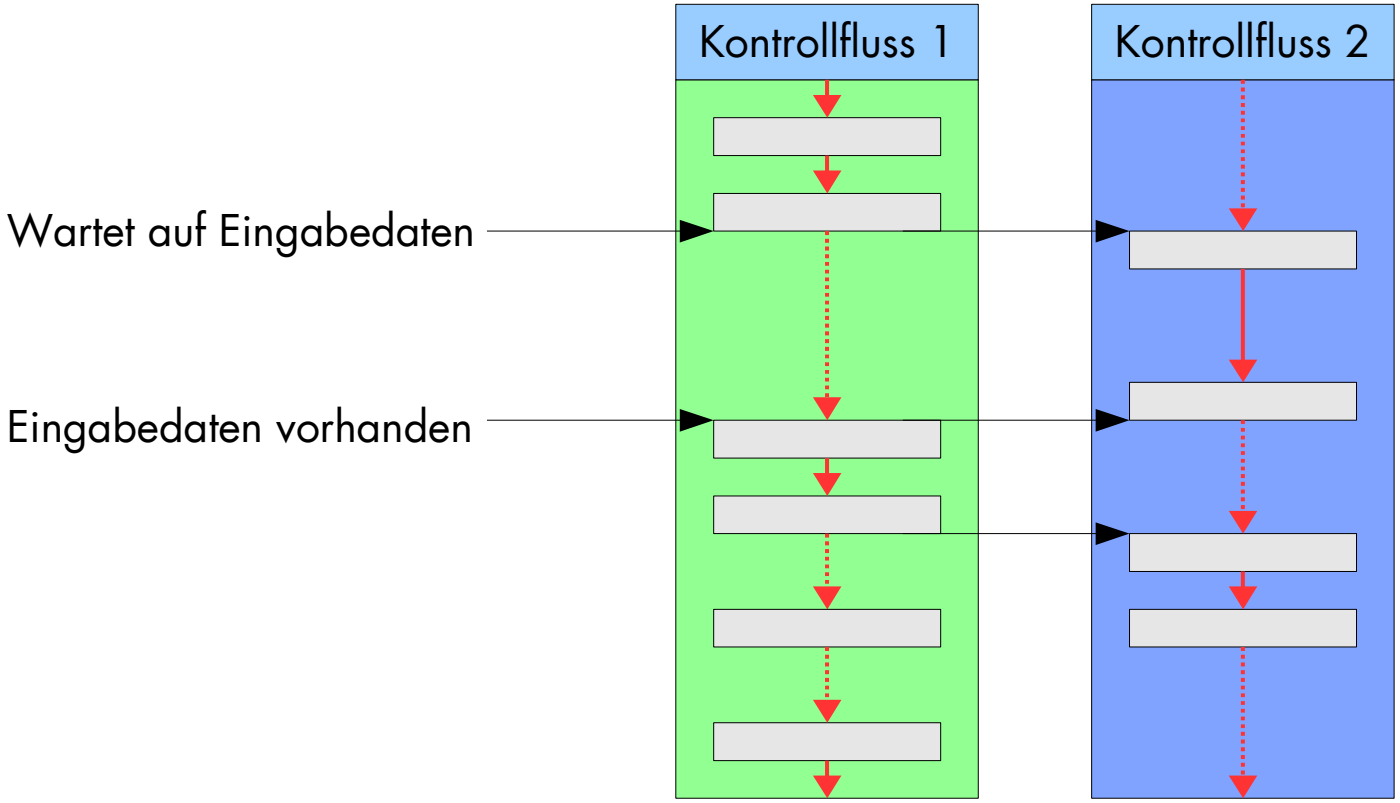
# Scheduler

- Teil des Betriebssystems zur Realisierung von Multitasking
- Steuerungsprogramm zur zeitlichen Ausführung der Kontrollflüsse
- Aufteilen zugewiesenen Rechenzeit auf die einzelnen Kontrollflüsse
- Präemptives Multitasking
  - Scheduler unterbricht laufenden Kontrollfluss
  - Übergibt die Kontrolle an einen wartende Kontrollfluss
- Kooperatives Multitasking
  - Kontrollfluss unterbricht laufende Verarbeitung
  - Übergibt die Kontrolle an den Scheduler
  - Scheduler gibt Kontrolle an wartenden Kontrollfluss

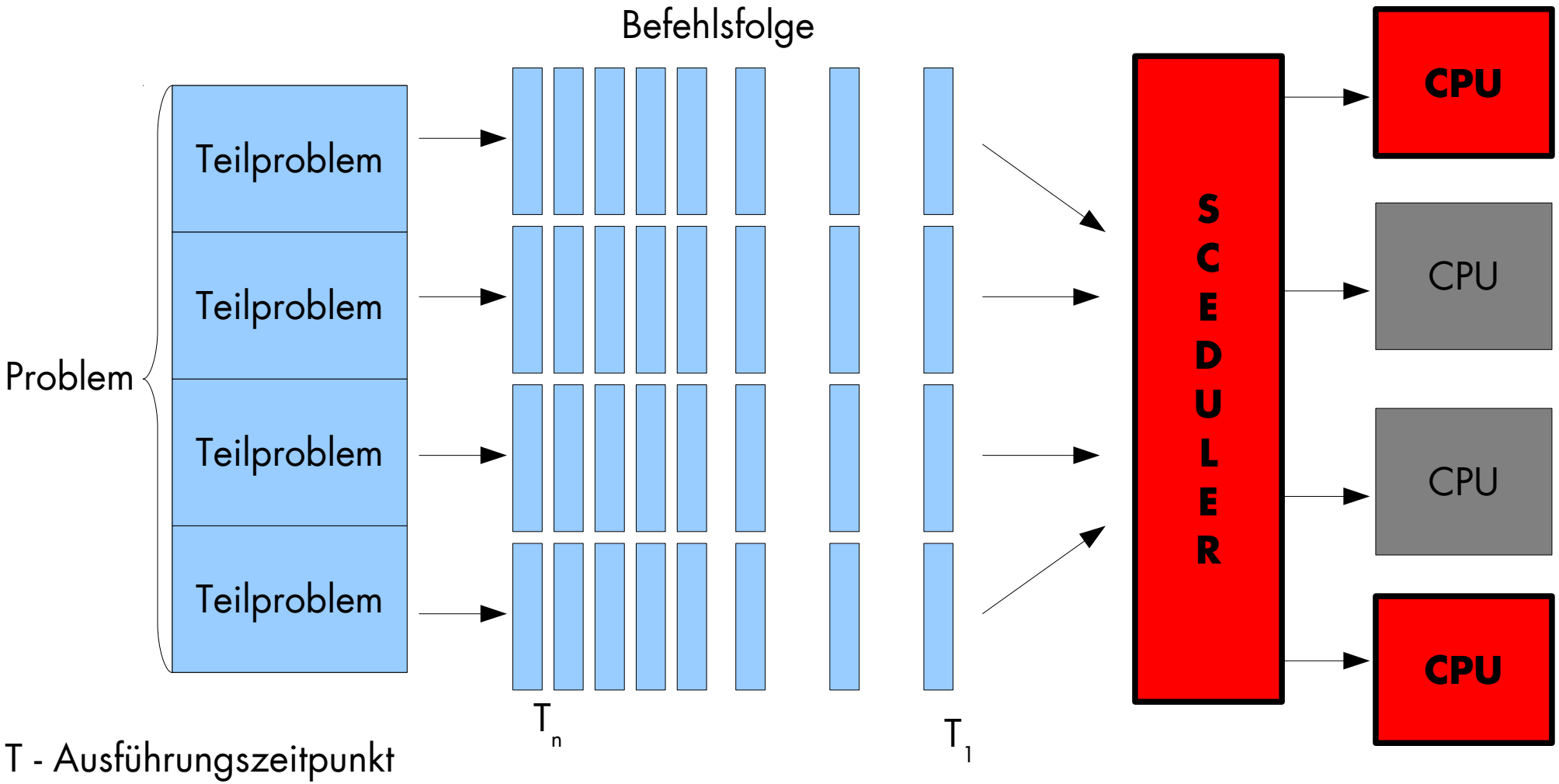
# Mehr Kontrollflüsse als Recheneinheiten



# Mehrere Kontrollflüsse Beispiel



# Scheduler als Verteilerinstanz

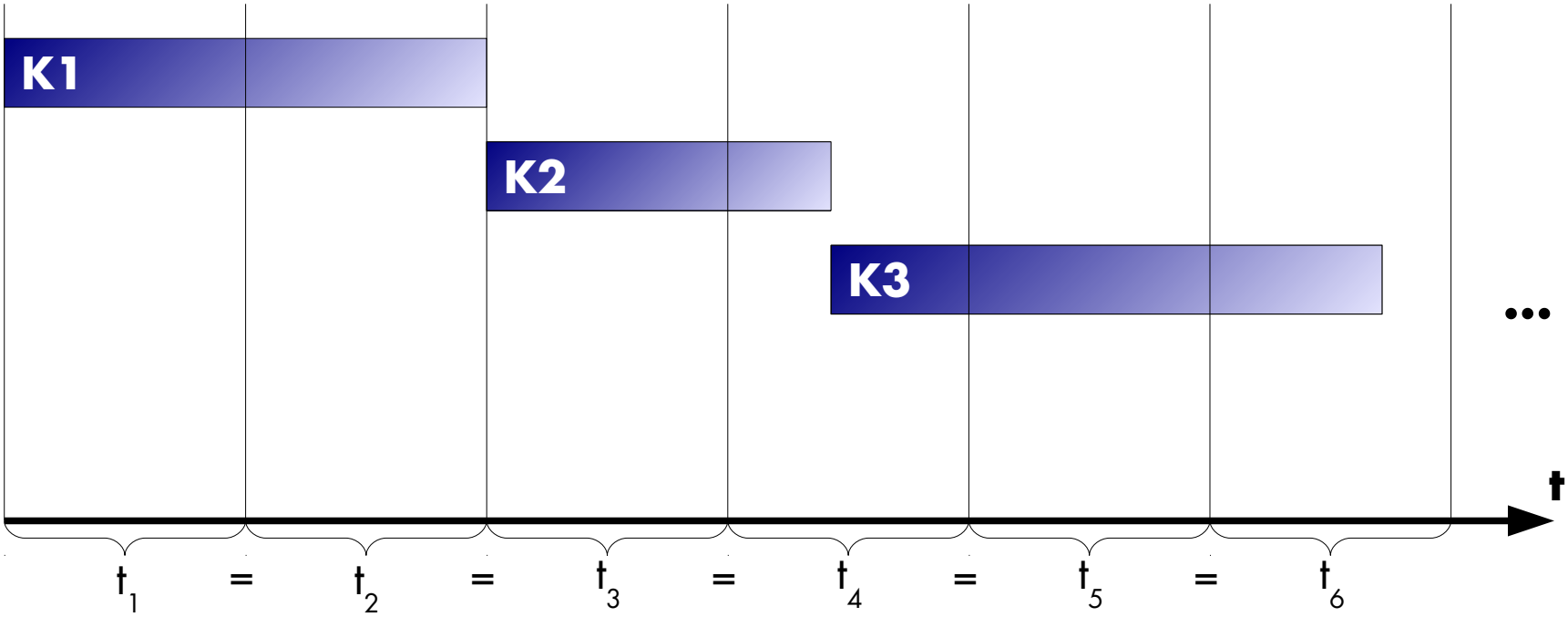


# Scheduling-Strategien

1) First Come First Serve (FCFS, auch FIFO)
- Abarbeitung der Prozesse nach Reihenfolge des Eingangs
- Einzelne Prozesse immer erst komplett verarbeitet
- Gute CPU-Auslastung
- Nicht optimale Auslastung von Speicherressourcen
- Für Mehrbenutzersysteme nicht geeignet

2) Shortest Job First (SJF, SPT)
- Kürzeste Jobs zuerst
- Rechenzeiten für einzelne Aufgaben müssen gut vorhersagbar sein
- Lange Aufgaben durch viele kurze Aufgaben ggf. nie bearbeitet

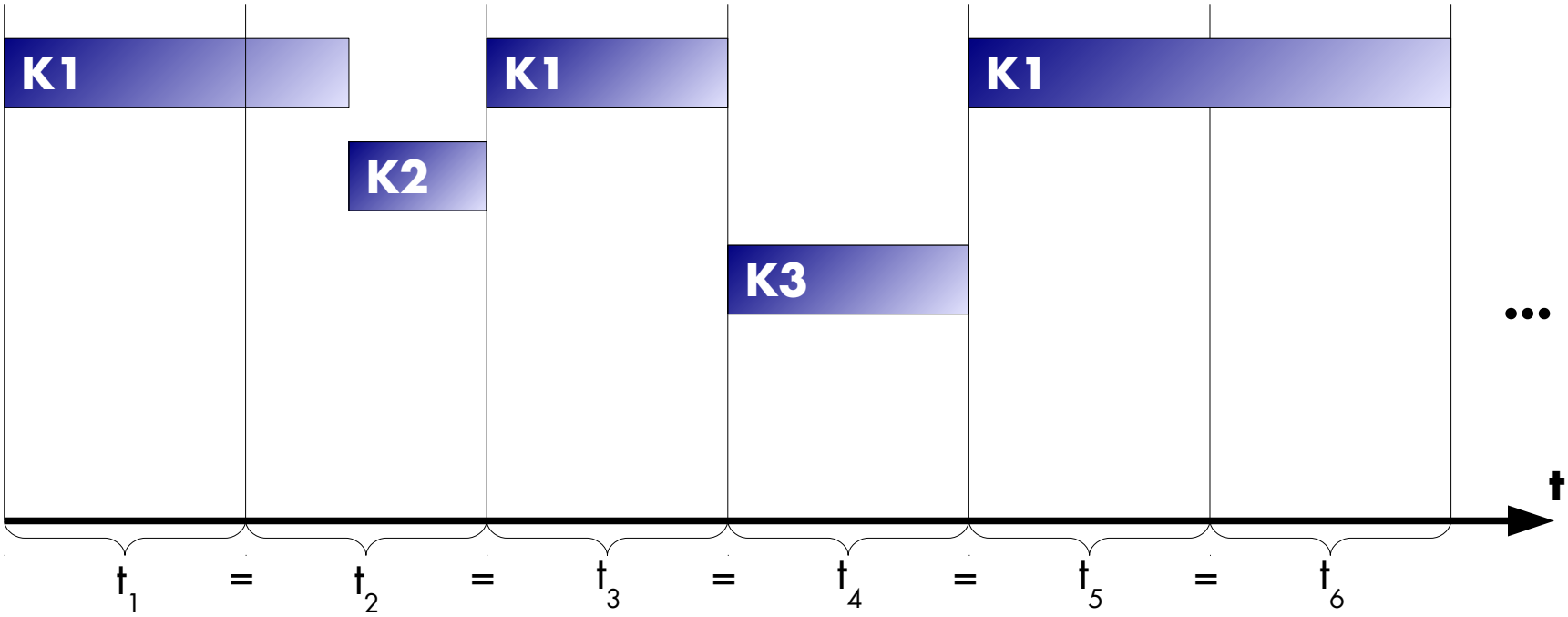
# 1) First Come First Serve



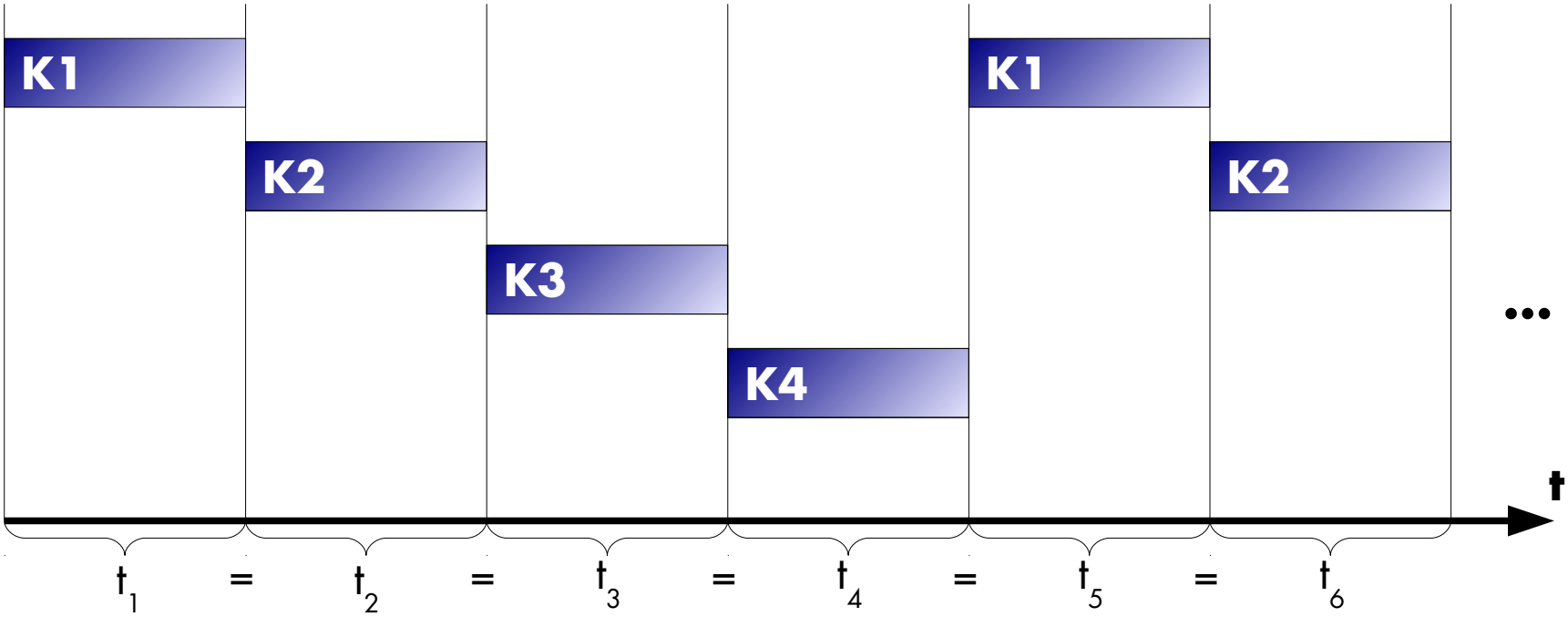
# Weitere Scheduling-Strategien

3) Shortest-Remaining-Time, Shortest-Remaining-Processing-Time
4) Earliest Due Date
5) Terminabhängige Ablaufplanung
6) Round Robin(Zeitscheibenverfahren)
7) Prioritätsscheduling

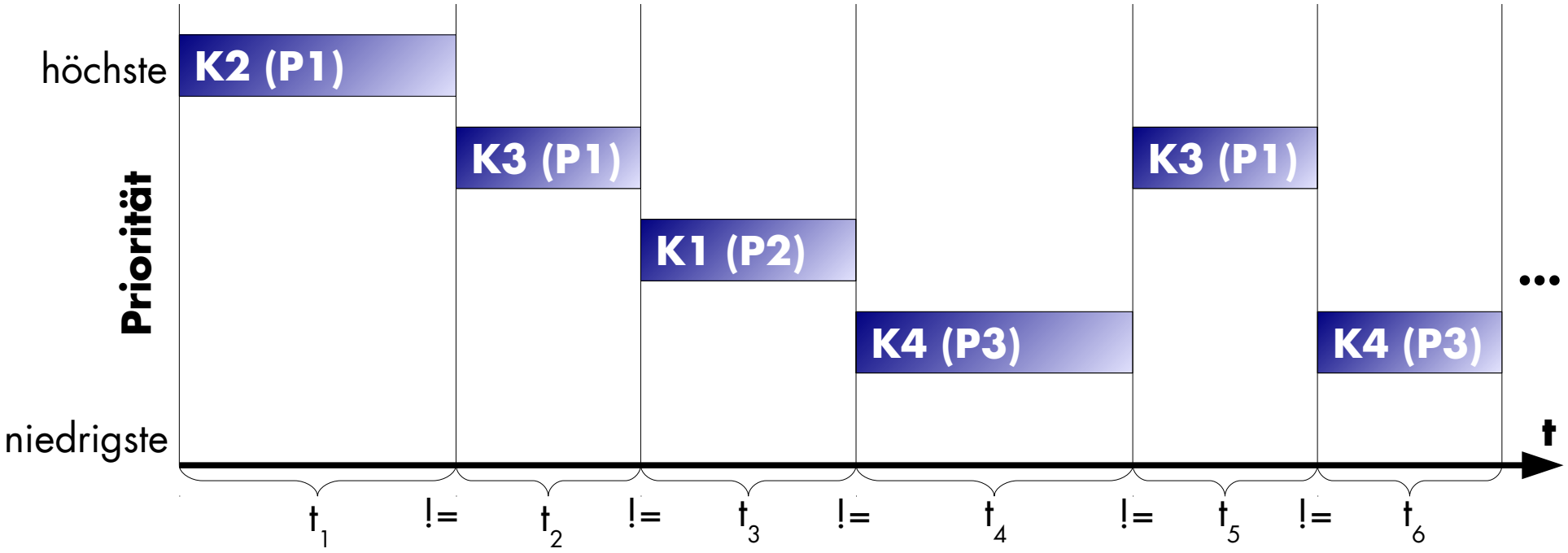
# 3) Shortest-Remaining-Time



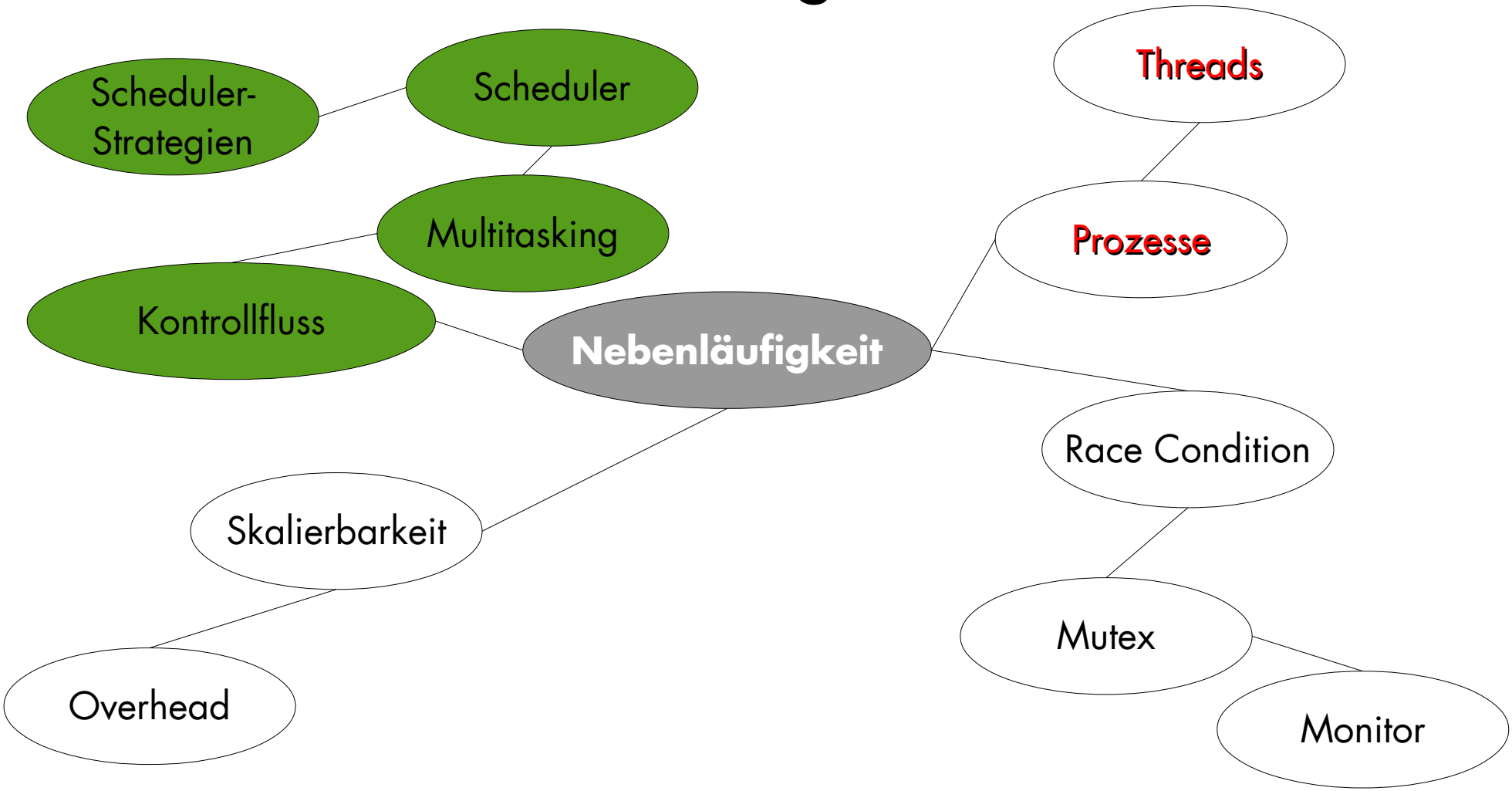
# 6) Round Robin – zeitgesteuerter Ablauf



# 7) Ablaufsteuerung nach Prioritäten



# Kontext der Nebenläufigkeit



# Prozesse vs. Threads

Threads (Thread kurz für: Thread of Control)

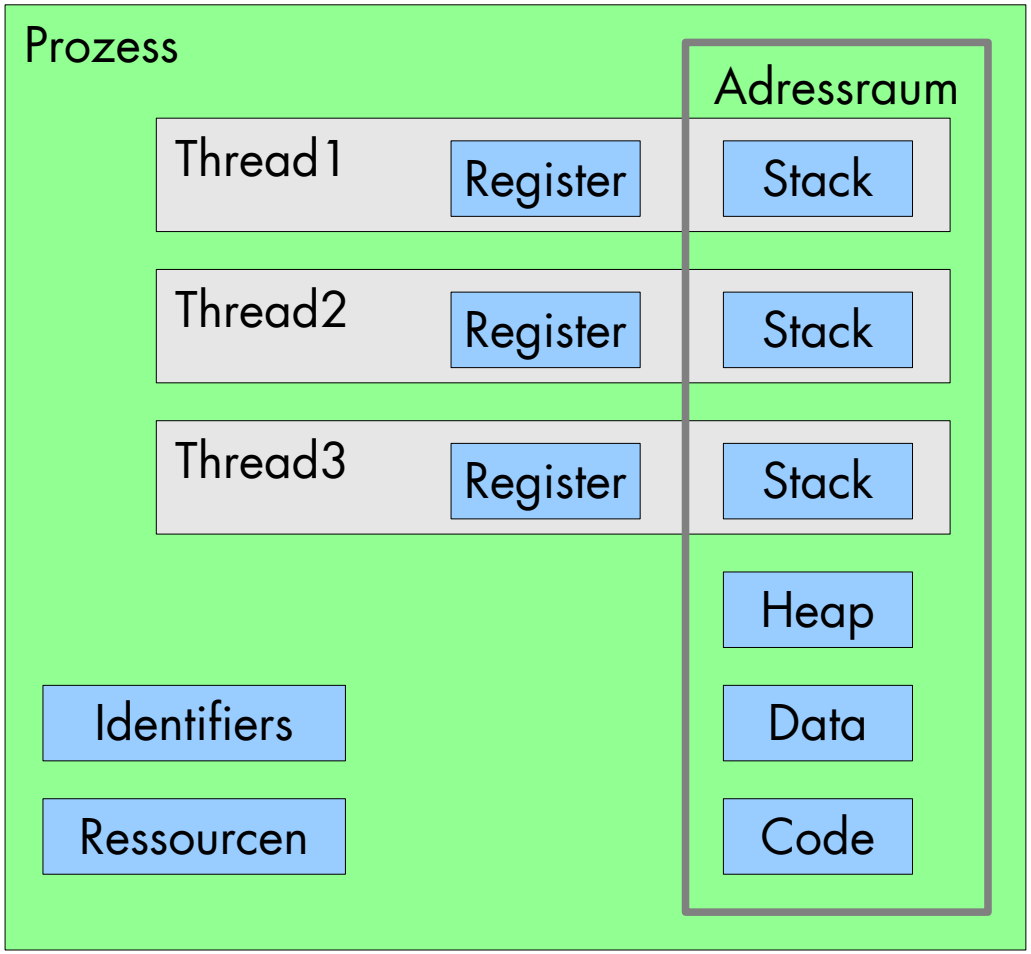
- Einzelner „Ausführungsfaden“ eines Programms, ein Kontrollfluss
- Threads existieren innerhalb von Prozessen
- Innerhalb eines Prozesses nicht isoliert, somit gemeinsame
  - Variablen (gemeinsamer Speicherbereich)
  - Betriebssystemabhängige Ressourcen (Dateien, Netzwerkverbindungen)
- Geringer Aufwand beim Erzeugen/Zerstören
- Kommunikation zwischen einzelnen Threads einfach
- Wechsel zwischen Threads innerhalb eines Prozesses einfach
- Gemeinsamer Speicher birgt jedoch Gefahr für Fehler

# Prozesse vs. Threads

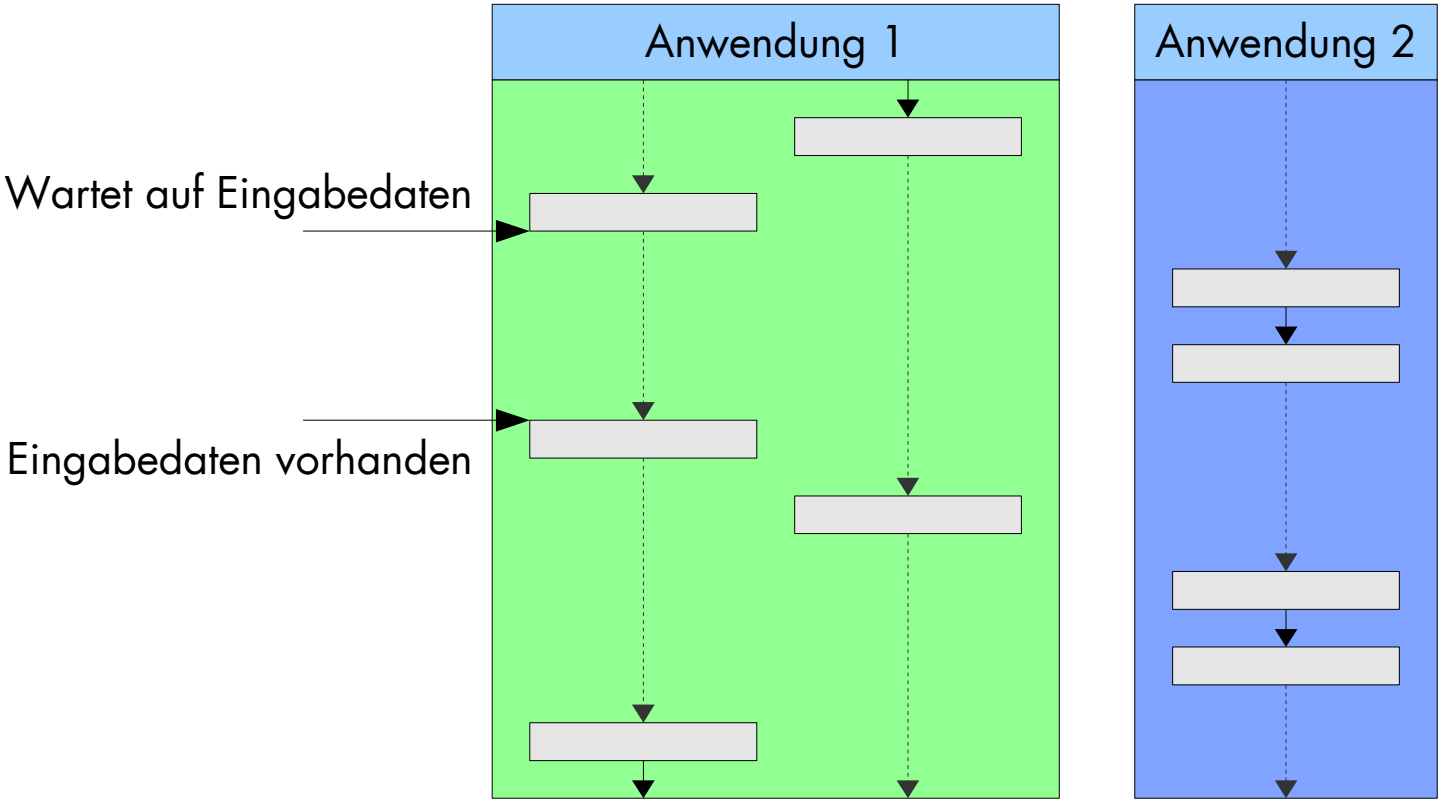
Prozesse (auch: Anwendungen oder engl. *Task*)

- Erzeugung, Ressourcen- und Ablaufkoordination durch Betriebssystem
- Werden vom Betriebssystem voneinander isoliert ausgeführt
- Vollständigen Satz eigener Variablen und exklusiver Speicherbereich
- Wechsel zwischen aktiven und nichtaktiven Prozessen aufwändig
- Kommunikation (Datenaustausch) mit anderen Prozessen – *Inter-Prozess-Kommunikation* - muss vom Betriebssystem realisiert werden

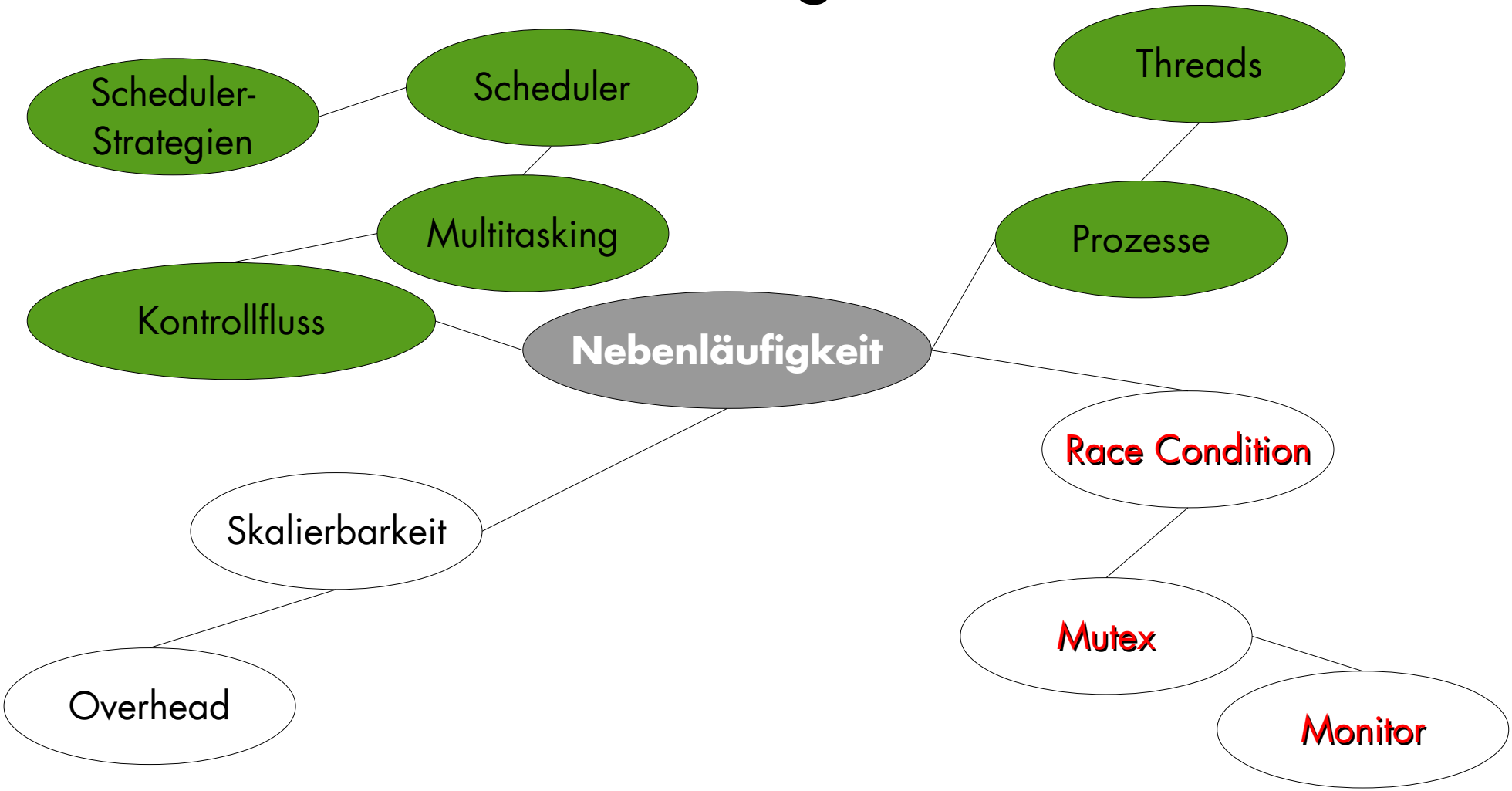
# *Threads* im Prozess



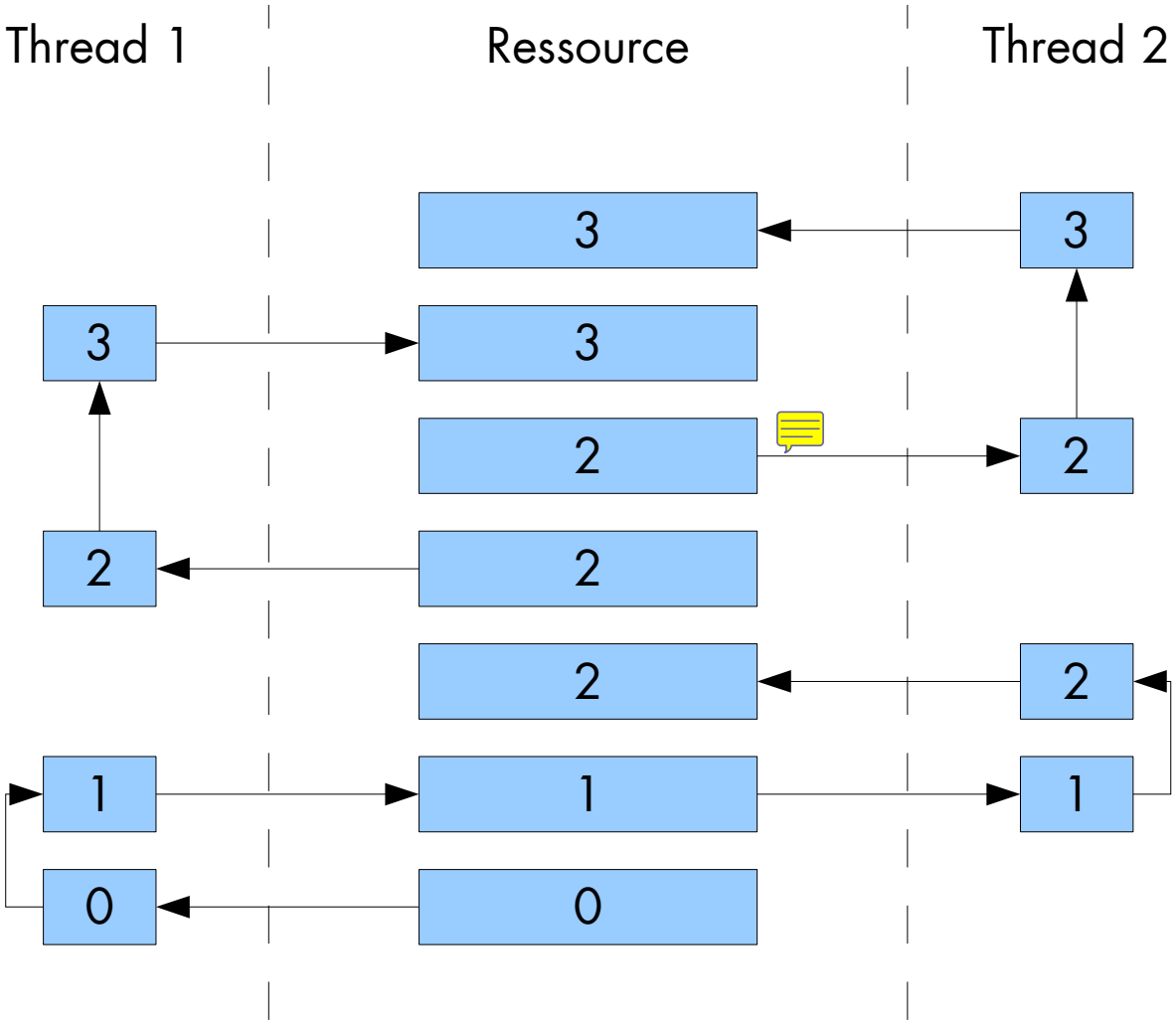
# Prozesse und *Threads*



# Kontext der Nebenläufigkeit



# Beispiel Race Condition



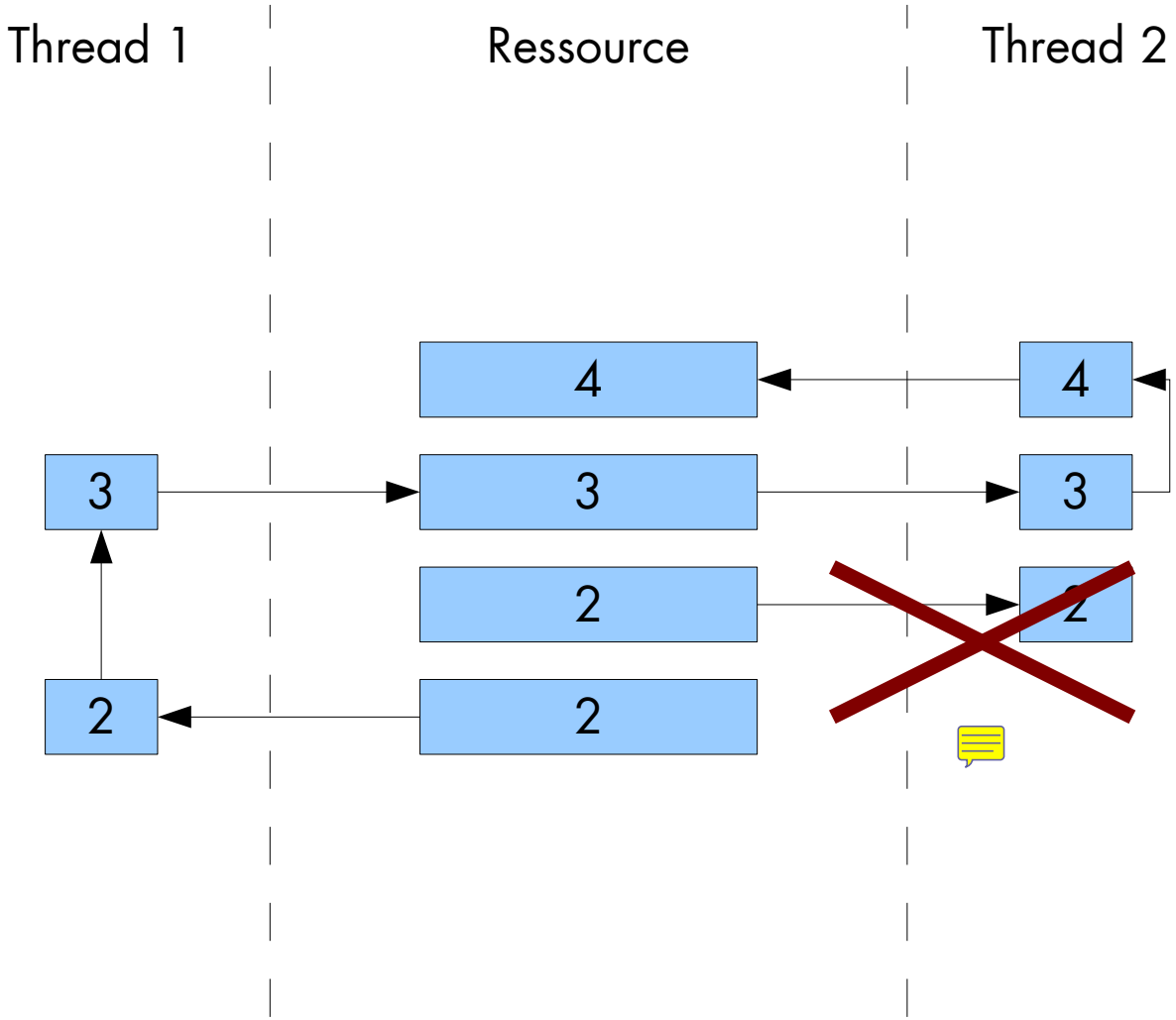
# Race Condition

- Wird auch als Wettlaufsituation bezeichnet
- Ergebnis einer Operation hängt vom zeitlichen Verhalten bestimmter Einzeloperationen ab
- Nicht deterministisches Verhalten tritt ein
- Häufig Ursache für schwer auffindbare Programmfehler
- Kann durch Einsatz eines Mutex vermieden werden

# Mutex (engl. Mutual Exclusion)

- Datenstruktur zur Synchronisation von Kontrollflüssen
- Vergleich Eisenbahntechnik: Signalanlage die anzeigt ob ein Streckenabschnitt frei ist und befahren werden kann
- Schützt einen bestimmten Bereich vor Mehrfachzugriff
- Kann ein Kontrollfluss den Bereich nicht betreten, wartet er
- Warten mehrere wird zufällig einer für nächste Ausführung bestimmt

# Beispiel Mutex



# Realisierungen eines Mutex (Auswahl)

Sperre (engl. *Lock*)

- Kontrollflüsse sichern sich vor Bearbeitung die Sperre und geben sie nach Bearbeitung wieder frei
- Sperren von Elementen, über Teile bis hin zur gesamten Ressource
- Ausprägungen: Lesesperre, Schreibsperre

Semaphor

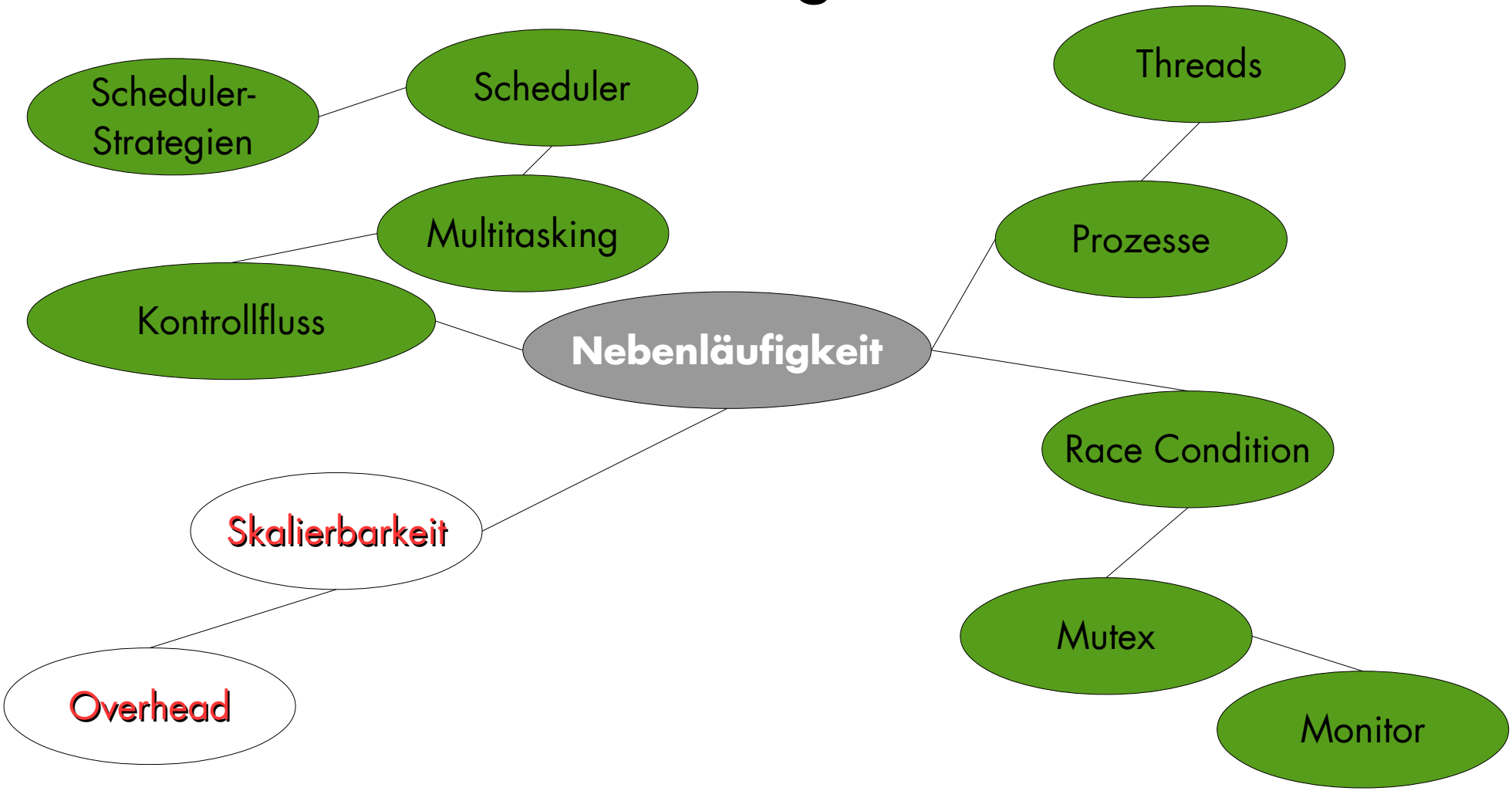
- Enthält zusätzlich Zähler für wartende Kontrollflüsse
- Reihenfolgebestimmung der wartenden Kontrollflüsse

# Realisierungen eines Mutex (Auswahl)

Monitor

- Synchronisationsmittel auf höherem Abstraktionsniveau
- Sprachliches Konzept zur Synchronisation von Zugriffen paralleler Kontrollflüsse auf gemeinsam genutzte Ressource
- Programmteile werden mit Elementen der Programmiersprache gekennzeichnet
- Keine Synchronisierungsanweisungen im Programmcode nötig
- Monitor sorgt selbstständig dafür, dass seine Monitor-Operationen immer nur von einem Prozess ausgeführt werden
- Verhindert zusätzlich Blockaden durch Kontrollflüsse, die nicht weiter geführt werden können mittels Bedingungsvariablen
- Kann mehrere Operationen umfassen

# Kontext der Nebenläufigkeit



# Skalierbarkeit

- Beschreibt Verhalten hinsichtlich wachsenden Ressourcennutzung
  - Mehr Ressourcen benötigt
  - Mehr Zugriffe auf Ressourcen
- Skalierbarkeit ist Verhalten zwischen Leistung und Ressourcen
  - Gute Skalierbarkeit: Hohe Leistungssteigerung bei überschaubarem Ressourcenzuwachs
  - Schlechte Skalierbarkeit: Überdurchschnittlicher Ressourcenzuwachs bei überschaubarer Leistungssteigerung
- Faustregel: Gut skalierbares paralleles Programm benötigt bei der doppelten Anzahl von Prozessoren in etwa die Hälfte der Rechenzeit
- Skalierbarkeit abhängig von Hardware **und** Software

# Skalierbarkeit

- Horizontale Skalierung
  - Verteilung von Funktionen auf zusätzliche Server
  - Es werden neue Ressourcen akquiriert
- Vertikale Skalierung
  - Ersetzen oder Ergänzen durch leistungsfähigere Ressource
  - Bestehende Ressourcen werden ausgewechselt oder aufgewertet

# Paralleler Overhead

- Zeit für die Koordinierung paralleler Verarbeitung (fehlende Nutzlast)
- Erzeugt Zusatzkosten und hat somit Auswirkung auf Skalierbarkeit
- Kann den Nutzen aus der Parallelisierung sogar übersteigen!
- Einflussfaktoren
  - Startzeiten von Aufgaben
  - Synchronisation
  - Datenkommunikation
  - Zeit um Aufgaben zu beenden

# Übersicht

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

# Probleme nebenläufiger Zugriffe

Verlorene Aktualisierungen (engl. *lost update*)

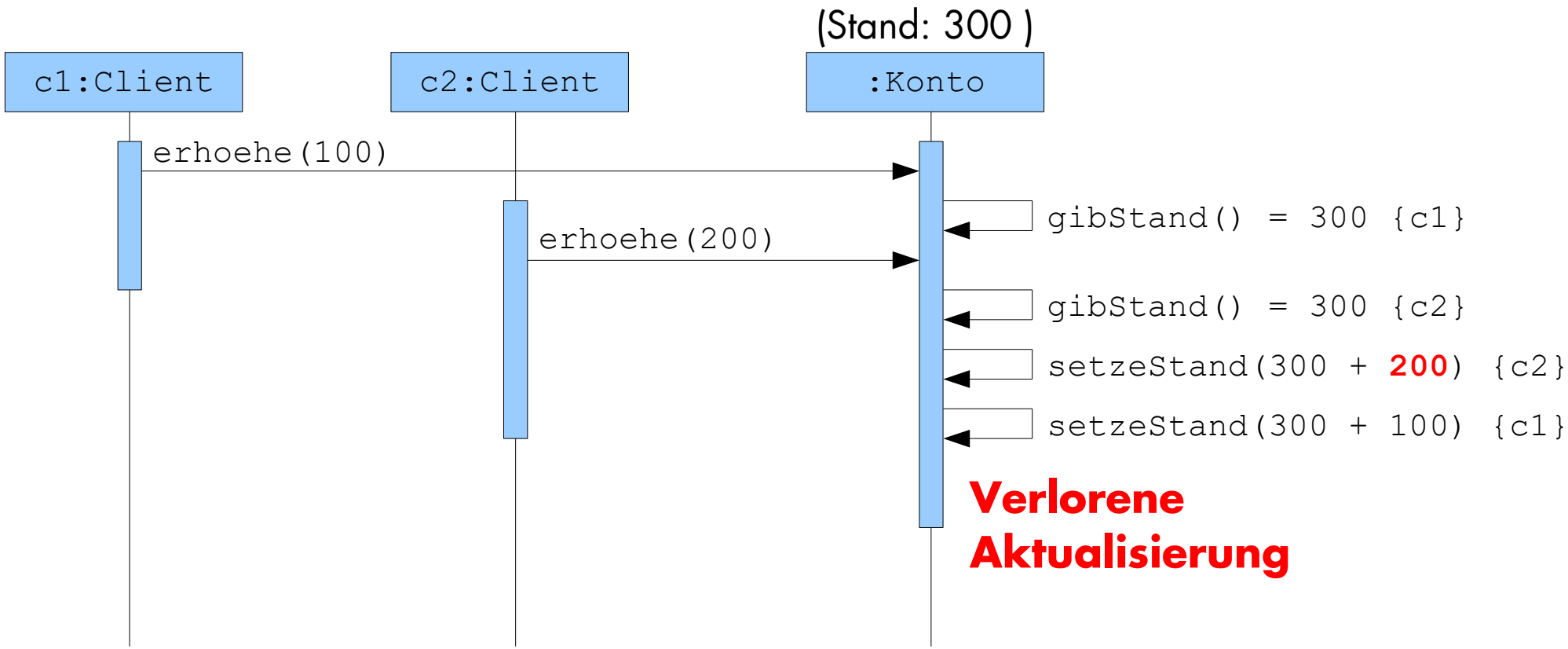
- Fehler bei parallelen Schreibzugriffen auf geteilte Information
- Zwei Kontrollflüsse verändern dieselbe Information
  - Erster ändert Information, ist aber noch nicht beendet
  - Zweiter überschreibt die Änderung
  - Änderung der ersten Transaktion geht verloren
- Eine Form der Race Condition

# Beispiel: Konto

| Konto |
| --- |
| - name<br>- stand |
| + erhoehe(betrag)<br>+ verringere(betrag)<br>+ gibStand()<br>- setzeStand(betrag) |

```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

# Verlorene Aktualisierung



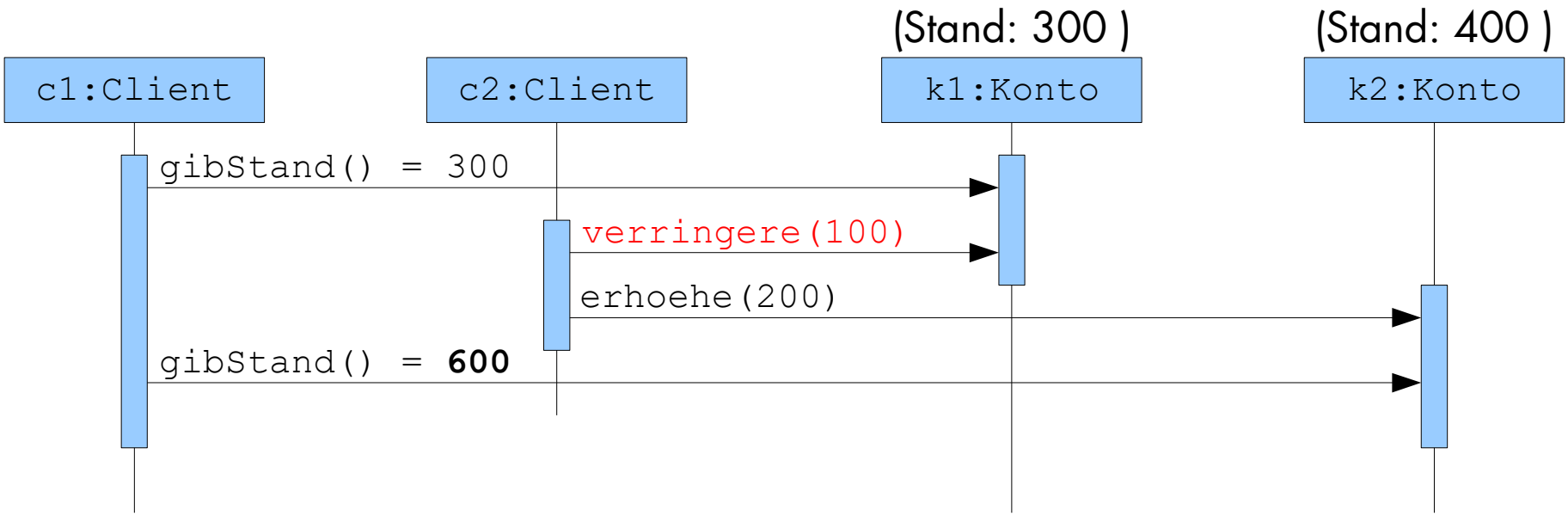

```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

# Probleme nebenläufiger Zugriffe

Inkonsistente Analyse (*inconsistent analysis*)

- Kontrollfluss erstellt Analyse aus mehreren Daten
- Daten werden während Analyse von zweitem Kontrollfluss verändert
- Erster Kontrollfluss registriert dies nicht
- Ergebnis ist falsche / nicht aktuelle Analyse

# Inkonsistente Analyse



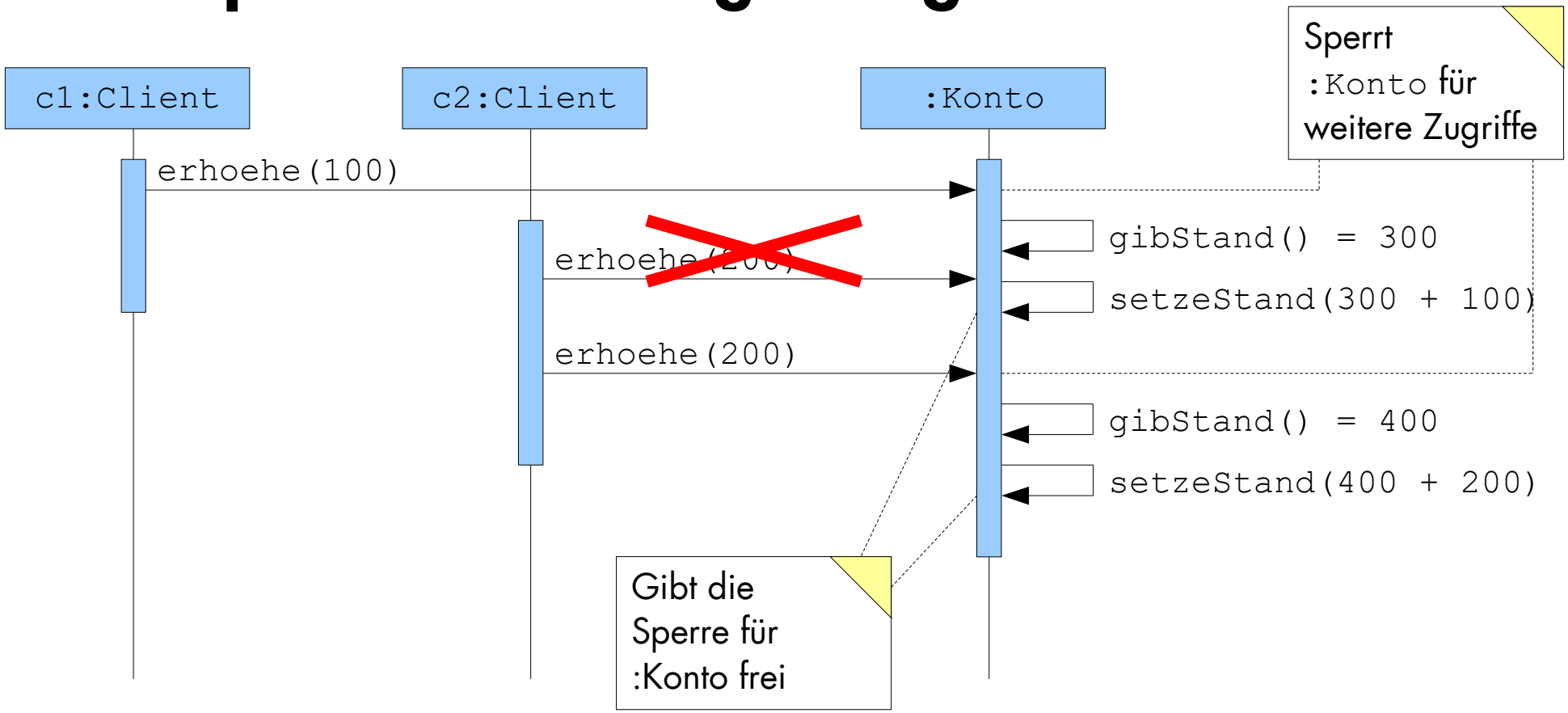

**Folge: Inkonsistentes Analyseergebnis bei Summenbildung**

```
Konto k1;
Konto k2;
...
int summe = 0;
summe = k1.gibStand() + k2.gibStand();
```

# Lösung Mutex

- Mutex zur Synchronisation mehrerer Kontrollflüsse bereits bekannt
- Kontrollstruktur zum Schutz eines **kritischer Bereichs**
- Führt zur Einschränkung der nebenläufigen Verarbeitung
- Kritischen Bereich so klein wie möglich halten
- Andere Kontrollflüsse müssen auf Freigabe des Bereichs warten
  - Aktives Warten (zyklische Abfrage)
  - Passives Warten („Schlafen“ bis er benachrichtigt wird)

# Beispiel: nebenläufiger Zugriff mit Mutex




```
void erhoehe(int betrag) {
    setzeStand(gibStand() + betrag);
}
```

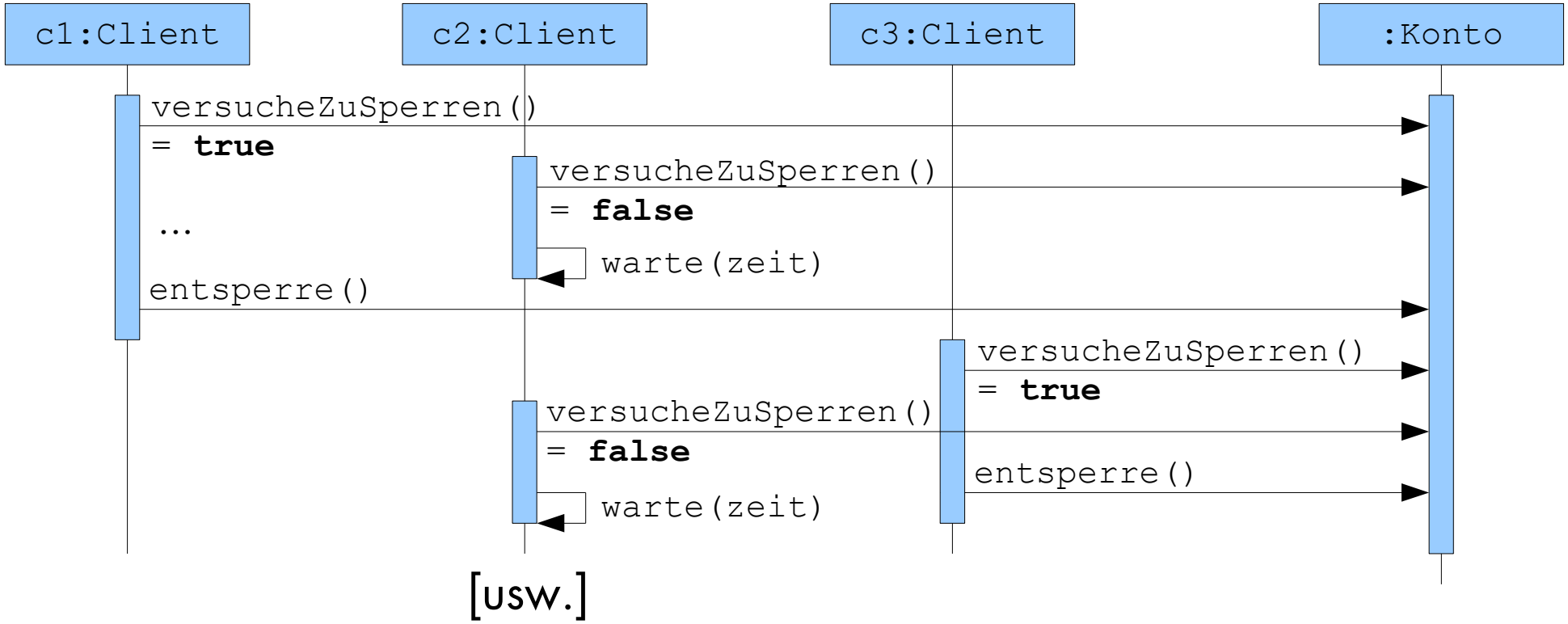
**Mutex**

# Probleme durch Mutex

Verhungernder Kontrollfluss

- Kontinuierlich warten mehrere Kontrollflüsse auf Mutex-Vergabe
- Beim Warten auf einem Mutex wird Kontrollfluss deaktiviert
- Mutex-Vergabe findet nicht deterministisch statt
- Zum Beispiel nur an aktive Kontrollflüsse oder rein zufällig
- Im Extremfall kann so ein Kontrollfluss nie zur Ausführung kommen
- Durch Einsatz von Scheduling-Algorithmen recht unwahrscheinlich

# Verhungernder Kontrollfluss

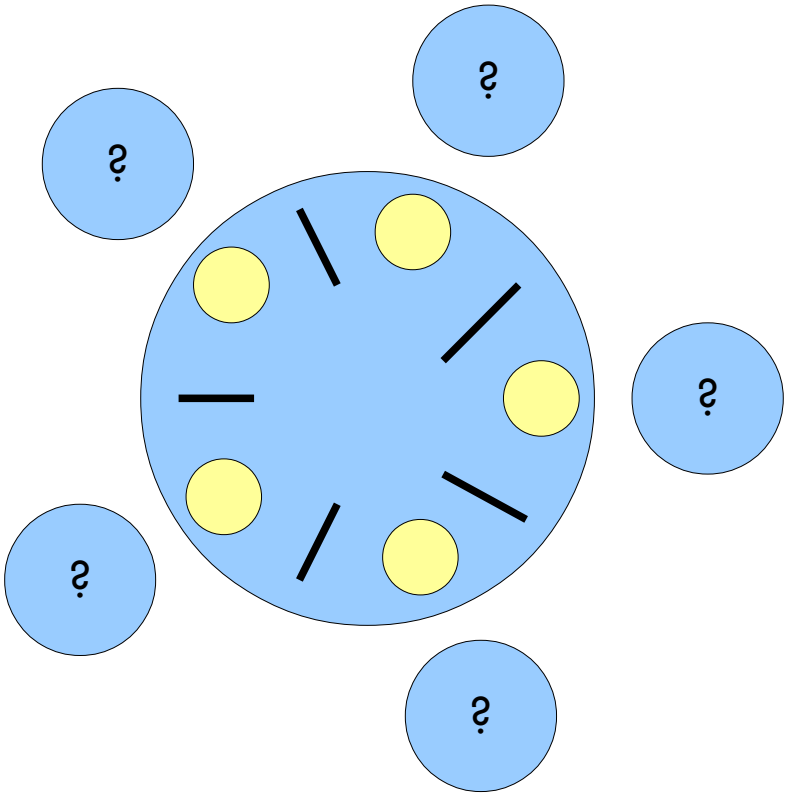


# Probleme durch Mutex

Verklemmungen (engl. *deadlocks*)

- Das gegenseitige Warten auf die Freigabe einer benötigten Ressource, ohne jedoch die eigene zuerst freizugeben
- Voraussetzungen
  - Mindestens zwei Kontrollflüsse sind im System aktiv,
  - Diese greifen auf dieselben(mindestens zwei) Ressourcen zu
  - Ressourcen werden durch *Mutex* synchronisiert
  - Kontrollflüsse sichern sich jeweils einen Teil der Ressourcen
  - Benötigen aber noch die weiteren Ressourcen zur Ausführung
  - Diese sind jedoch bereits durch andere Kontrollflüsse reserviert
  - Freigabe der Ressourcen erfolgt ausschließlich kooperativ(?)

# Philosophenproblem (Verklemmung)




- Nachdenken
- Hunger
  - Linkes Stäbchen nehmen
  - Rechtes Stäbchen nehmen
- Essen
- Beide Stäbchen zurücklegen

# Philosophenproblem (Verklemmung)

- Fünf Philosophen sitzen am Tisch und denken über philosophische Probleme nebenläufiger Systeme nach
- Wird einer hungrig so benötigt er zwei Stäbchen zum Essen
- Er nimmt zunächst das linke, dann das rechte Stäbchen
- Sobald er satt ist, legt er beide Stäbchen wieder zurück und denkt wieder über philosophische Probleme nebenläufiger Systeme nach
- Ist ein Stäbchen nicht an seinem Platz so wartet der Philosoph bis es wieder verfügbar ist
- Hat er bereits ein Stäbchen in der Hand, legt er es nicht zurück
- Parallele Kontrollflüsse (Philosophen) sind auf gemeinsame Ressourcen (Stäbchen) angewiesen

# Philosophenproblem (Verklemmung)

- Funktioniert sehr gut, sofern nur einzelne Philosophen hungrig sind
- Falls alle Philosophen zur gleichen Zeit hungrig sind wird es allerdings problematisch
  - Alle ergreifen zuerst das linke Stäbchen
  - Nehmen dem links sitzenden Kollegen sein rechtes Stäbchen weg
  - Alle warten anschließend darauf, dass das rechte Stäbchen auftaucht
  - Da keiner der Philosophen das linke Stäbchen wieder zurücklegt, verhungern alle Philosophen
- Vergebliches Warten auf Ereignis das aufgrund der Blockade nie eintrifft - **Verklemmung/Deadlock**

# Verhindern von Verklemmungen

- Ein Kontrollfluss gibt bereits gesicherte Ressourcen frei
  - Wenn er einen benötigten *Mutex* nicht erhält
  - *Mutex* kann auch von übergeordneter Stelle entzogen werden (Monitor)
  - Aufwändig, speziell wenn viele Ressourcen gesperrt werden müssen
- Jeder Kontrollfluss gibt vorher an, welche Ressourcen er benötigt
  - Wenn diese alle zum selben Zeitpunkt frei sind, bekommt der Kontrollfluss für alle Ressourcen den *Mutex*
  - Kann sehr lange dauern – Verhungernder Kontrollfluss
- Ressourcen werden durchnummeriert und nur in aufsteigender Reihenfolge vergeben
- Auflösung des *Mutex* – benötigte Ressourcen für alle Prozesse wieder zugänglich machen

# Beseitigung von Verklemmungen

- Prozessabbruch
  - Einfachste Variante
  - Gute Auswahl des Prozesses wichtig – schlecht automatisierbar
  - Datenverlust nicht vermeidbar nur einschränkbar
- Präemption
  - Prozess unterbrechen und erst später fortsetzen
  - Zwischenzeitlich arbeiten die blockierten Prozesse ihre Aufgaben ab
- Rollback
  - Sicherungen jedes Prozesses vorher notwendig
  - Im Falle eines Verklemmung wird ein Rollback des betroffenen Prozesses auf gesicherten Stand vorgenommen
  - „Neustart“ des Prozesses sobald Ressourcen wieder frei sind

# Übersicht

- Serielle und parallele Verarbeitung
- Kontext der Nebenläufigkeit
- Probleme nebenläufiger Systeme
- Theoretische Modelle zur Nebenläufigkeit

# Allgemein

- Zur Konzeption nebenläufiger Systeme eignen sich technisch unabhängige theoretische Modelle
- Erlauben einen Überblick und ein Verständnis der Abläufe mit Fokus auf Nebenläufigkeit
- Auswahl
  - Parallel Random Access Machine (PRAM)
  - Bulk synchronous parallel (BSP)
  - Simple Concurrent Object-Oriented Programming (SCOOP)
  - Linda
  - Petri-Netz

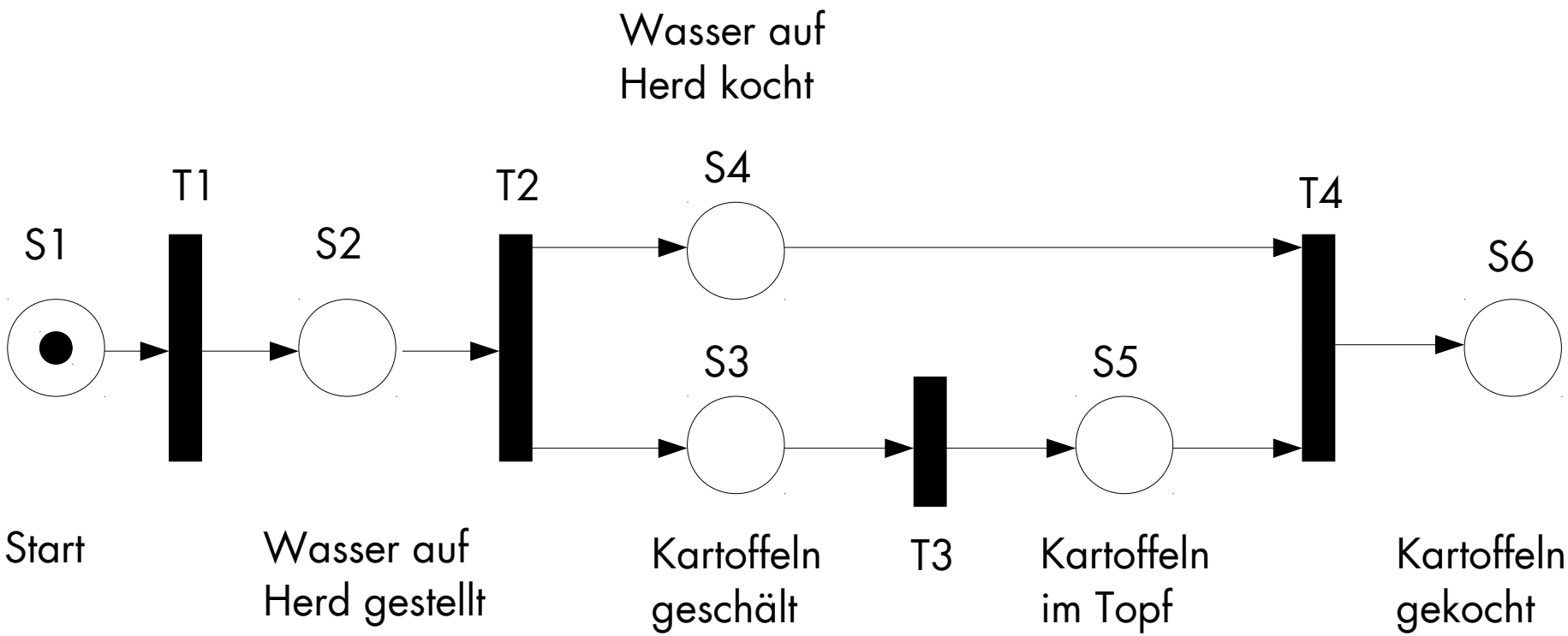
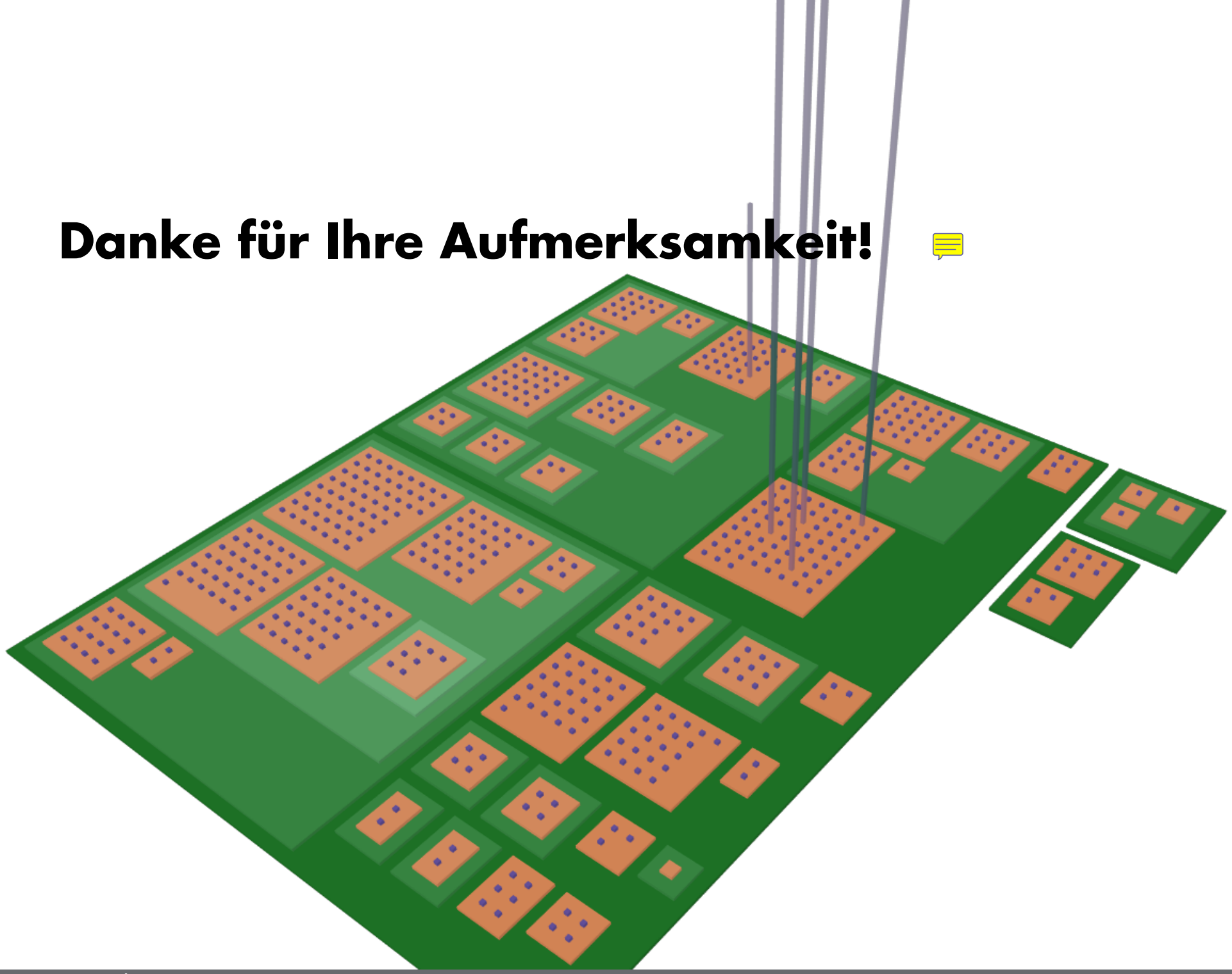
# Petri-Netz

- Entwickelt durch Carl Adam Petri als mathematisches Modell nebenläufiger Systeme
- Formale Methode zur Modellierung von Transformationsprozessen
- Verallgemeinerung der Automatentheorie
- Petri-Netz ist ein gerichteter Graph, bestehend aus:
  - Stellen (engl. *Places*)
  - Transitionen (engl. *Transitions*) Übergängen zur Informationsverarbeitung
  - Gerichtete Kanten miteinander verbundener Elemente
- Keine direkten Verbindungen zwischen zwei Stellen oder Transitionen

# Funktionsweise

- Stellen besitzen Kapazität und enthalten entsprechend viele Token
- Kanten besitzen Gewichte, die Kosten entsprechen
- Kapizitäten und Gewichte sind per default 1
- Transitionen werden als schaltbereit bezeichnet wenn:
  - An Eingangstellen mindestens so viele Token vorhanden sind, wie Kosten von der Transition verursacht werden
  - Ausgangsstellen genügend Kapazität haben um Token aufzunehmen
- Belegung der Stellen gibt den Zustand des Petri-Netz an

# Beispiel Kartoffeln kochen



# Danke für Ihre Aufmerksamkeit!